

Q-Trust

Senior-Engineer Codebase Audit

Cross-organizational PQC migration coordinator on Base L2

Repository `github.com/humoge7502/q-trust`
Commit `ec67f64 "refactor: reorganize project structure"`
Audit Date `2026-08-22`
Scope `Full repository (every tracked file inspected)`
Auditor `Z.ai Senior Engineer Audit`

1. Executive Summary

What the project is

Q-Trust is a cross-organizational protocol for coordinating the migration of classical cryptography (RSA, ECC, DSA, DH) to post-quantum cryptography (ML-KEM, ML-DSA, SLH-DSA, HQC, FALCON) on the Base L2 Ethereum rollup. The architecture follows a deliberately minimal-on-chain design: only SHA-256 hashes of Cryptographic Bills of Materials (CBOMs) live on-chain, while full CBOMs and evidence artifacts are stored off-chain on IPFS via Pinata. The system is decomposed into Solidity registry contracts (Foundry project), a Python SDK (web3.py + eth-account), a Python cryptography scanner that produces CBOMs from live TLS/SSH scans, a PyTorch-Geometric GNN planner that ranks migration priority, a Fastify + viem backend that exposes a REST API and an EIP-712 gasless relay, and a Next.js 16 frontend with public verification, an org dashboard, and a vendor portal. The repository currently contains 9 Solidity contracts behind TransparentUpgradeableProxy + UUPS, of which 5 (AssetRegistry, VendorRegistry, MigrationRegistry, AuditRegistry, QTrustGovernance) are actively used and 4 (RevocationAnchor, PolicyCommitment, SchemaRegistry, TrustAnchorRegistry) are deployed but unused by the backend.

The repository was generated by an LLM (per docs/PROMPT.md, "Master Build Prompt for Qwen 2.5") and has 5 commits on the default branch with no tags, no PRs, no issues, and no CI workflows. The git history shows an initial commit, a "P0 critical fixes from assessment" commit, a feature commit that added trust infrastructure contracts and the VC stack, a deployment/SDK fix commit, and a final structural refactor. The most recent commit (ec67f64) is a reorganization, not new functionality.

Apparent goals and intended users

The intended users are organizations performing PQC migration (banks, credit unions, enterprises), vendors of cryptographic products (DigiCert, Thales, AWS), and third-party auditors. The protocol aims to provide cryptographic provenance: an organization registers its CBOM hash on-chain, vendors attest which product versions support which PQC algorithms, the organization records each migration step with evidence, auditors post audit attestations bound to on-chain state, and any third party can verify the full chain of trust via a public URL (e.g., /v/<asset-id>). The patent materials in docs/PATENT/ indicate a commercial intent: SaaS subscription revenue (\$1K/mo Pro, \$10-50K/mo Enterprise), with no token and no speculation. The PROMPT.md design notes mention an ERC-4337 paymaster so non-crypto customers can pay in fiat via Stripe — but this paymaster is not implemented anywhere in the repository.

Overall architecture and maturity

The architecture is layered: client (frontend + SDK + CLI inspector) → API/relay (Fastify + viem) → services (Postgres indexer, Redis, GNN planner microservice) → on-chain (Base L2 registries + governance). This is a sensible separation of concerns. However, the implementation is at the late-prototype / early-development stage. The smart contracts are well-designed and thoroughly tested (127 Forge tests across 10 suites including adversarial cases). The backend has working read and write endpoints and an EIP-712 gasless flow that is properly signed and replay-protected. The Python SDK has a full E2E test against local anvil. But the backend has zero automated tests, the frontend has zero automated tests, there is no CI/CD pipeline, no monitoring, and several endpoints are stubs that return hardcoded values (see § 16). The deployment has never gone beyond local anvil.

Major strengths

- OpenZeppelin AccessControl + TransparentUpgradeableProxy + UUPS upgradeability + TimelockController governance — the contract layer uses industry-standard, audited primitives.

- EIP-712 gasless attestation flow: vendors sign typed data off-chain, the relay verifies the signature and submits on-chain, the contract records the SIGNER as the vendor. Vendors never need to hold gas or run a node.
- Anti-replay via on-chain nonces checked both in the relay and the contract (AssetRegistry.sol:136, VendorRegistry.sol:156, MigrationRegistry.sol:156).
- Anti-SSRF in webhook delivery (webhook.ts:22-46) — blocks localhost, 10/8, 172.16-31, 192.168/16, .local, .internal, requires HTTPS.
- Cross-registry integrity: MigrationRegistry calls AssetRegistry.verifyAsset before recording (MigrationRegistry.sol:222); AuditRegistry rejects assetsMigrated > on-chain migration count (AuditRegistry.sol:87-90).
- Honest benchmark reporting in the README — earlier inflated GNN claims ("Kendall τ 0.924") were retracted and replaced with multi-seed mean $\pm$ std.
- Idempotent indexer upserts via INSERT ... ON CONFLICT DO UPDATE; graceful degradation to direct RPC reads when Postgres is unavailable.

Major weaknesses

- Several backend endpoints are stubs that return hardcoded values. /v1/credentials/verify (server.ts:503-521) returns {valid:true, revoked:false} for ANY input — this is actively misleading. /v1/revocation, /v1/policies, /v1/schemas, /v1/trust-anchors return hardcoded 0x00 hashes.
- Dev hole: when QTRUST_API_KEYS is unset, requireApiKey (server.ts:99-102) calls done() and allows all /v1/write/* routes including /v1/write/assets — the API key gate is opt-in rather than fail-closed.
- Webhook fanOut is exported but NEVER called anywhere in the codebase (grep-confirmed). The subscribe/unsubscribe endpoints maintain Redis sets that no code reads. The webhook delivery path is controlled entirely by the QTRUST_WEBHOOKS env var (a comma-separated URL list). The Redis-backed subscription system is non-functional theater.
- BullMQ is listed in backend/package.json:17 but is never imported in any source file. The webhook service is described in the README as "BullMQ webhooks" — this is inaccurate.
- v2 GNN (planner/qtrust_planner/model_v2.py) is a 175-line "patent-grade hybrid GCN+GATv2+centrality" model that is referenced in patent materials but never used by train.py, predict.py, or server.py — all three import only the v1 GCN (model.py). v2 is dead code.
- VCVerifier.verify_credential (sdk/qtrust/vc.py:319-379) does NOT verify the Ed25519 signature. Line 354 comment says "Signature verification delegated to holder" — meaning the verifier returns valid=true without cryptographically checking the proof.
- QTrustGovernance.schedulePause (QTrustGovernance.sol:74-79) and scheduleUnpause always encode AssetRegistry.pause signature regardless of the registryIndex argument. This works only because all four registries inherit Pausable and have identical pause() signatures; it would fail for any future contract with a different signature.
- EIP-712 domain chainId is hardcoded to 84532 (Base Sepolia) in attestation.ts:127, 233, 334, even though config.ts supports QTRUST_USE_MAINNET=true (chainId 8453). The relay walletClient (attestation.ts:36) is also hardcoded to baseSepolia.
- No CI/CD. The repository has no .github/workflows/, no GitLab CI, no Jenkinsfile. The only CI-like artifact is scripts/verify_all.sh, intended to be run manually. No push triggers it.
- Planner Dockerfile (planner/Dockerfile:10) COPYs model.pt — but .gitignore:28 excludes model.pt — so a fresh clone cannot build the planner image.

- Anvil dev private key (0xac09...ff80) appears in 6 committed files including backend/.env.example and pilot/run_pilot.py. This is the well-known anvil dev key (not real funds), but committing it in .env.example is a pattern anti-recommendation.

2. Repository Structure

The repository is organized as a polyglot monorepo with one top-level directory per logical component. Below is a directory-by-directory walkthrough with the entry points and execution flows.

contracts/ — Solidity registries (Foundry)

Foundry project at contracts/ with foundry.toml configuring solc 0.8.24, optimizer enabled with 200 runs, no via_ir, and remappings to OpenZeppelin and forge-std submodules. The src/ directory contains 10 Solidity contracts: 4 core registries (AssetRegistry, VendorRegistry, MigrationRegistry, AuditRegistry), 1 governance facade (QTrustGovernance), and 4 trust-infrastructure contracts (RevocationAnchor, PolicyCommitment, SchemaRegistry, TrustAnchorRegistry). All contracts inherit OpenZeppelin AccessControl, Pausable, Initializable, UUPSUpgradeable; the three write registries also inherit ReentrancyGuard. test/ contains 10 test files including Attack.t.sol (adversarial cases). script/Deploy.s.sol is the deployment entry point — it deploys all 9 contracts behind TransparentUpgradeableProxy, wires MigrationRegistry → AssetRegistry and AuditRegistry → MigrationRegistry, grants REGISTRAR/MIGRATOR/AUDITOR roles to deployer, hands DEFAULT_ADMIN_ROLE to a 2-day TimelockController, deploys QTrustGovernance as a facade, then deploys the 4 trust-infra contracts (also handed to the timelock).

backend/ — Fastify + viem API, relay, webhook service

TypeScript ESM project (type: module) with package.json targeting NodeNext module resolution and strict: true. The src/ tree is small: server.ts (the main file with all route definitions), config.ts (viem publicClient + env-var extraction + parseAssetId), db/schema.sql (Postgres DDL for the indexer read model), lib/abis.ts (62KB of auto-generated ABI bindings — a code smell), and services/{attestation,indexer,verify,webhook}.ts. Entry point: src/server.ts, started via `node dist/server.js` after `tsc -p tsconfig.json`. The Dockerfile uses oven/bun:1.1 as base and falls back to bun runtime — a mismatch with the npm scripts that reference tsc directly.

frontend/ — Next.js 16 + React 19 + viem

Next.js 16 application with React 19, viem 2.16 for blockchain reads, @tanstack/react-query 5 for data fetching, reactflow 11 (deprecated — should be @xyflow/react) for the provenance graph, Tailwind CSS 4, clsx, tailwind-merge. Pages: src/app/page.tsx (home), /v/[id]/page.tsx (public verification page), /dashboard/page.tsx (org dashboard), /vendors/page.tsx (vendor portal), /v/page.tsx. Components: attestation-form.tsx (gasless EIP-712 form), dynamic-provider.tsx (wallet context — real MetaMask first, mock fallback), planning-panel.tsx, query-provider.tsx. Hook: use-user-role.ts (infers role from whether the address has assets/attestations). API route: /api/vendor-registry-address/route.ts (exposes the VendorRegistry proxy address for browser EIP-712 signing). next.config.mjs rewrites /api/* to the Fastify backend.

sdk/ — Python SDK (web3.py + eth-account + pydantic + httpx)

Installable Python package (pyproject.toml, pip install -e .) with dependencies: web3>=6.0, eth-account>=0.10, pydantic>=2, requests, python-dotenv. Optional dev deps include pytest, ruff, mypy strict. Modules: client.py (QTrustClient — full contract wrapper, EIP-712 signers), contracts.py (auto-generated ABIs), did.py (did:web resolver with 5-min cache), ipfs.py (Pinata pinning client), schema.py (Pydantic models: CBOM, CBOMEntry, AssetRecord, ProductAttestation, MigrationRecord, VendorInfo), trust.py (TrustAssessment + TrustEvaluator — deterministic policy engine with weighted clauses), vc.py (W3C VerifiableCredential + VCIssuer + VCPresenter + VCVerifier — but see § 5 for the verifier bug). Tests: test_client.py, test_did.py, test_trust.py, test_vc.py, and e2e_anvil.py (full lifecycle

against local anvil).

inspector/ — cryptography-inspector CLI

Installable Python package (crypto-inspector CLI) with dependencies: cryptography>=41, python-nmap, pydantic, typer, rich. Modules: scanner.py (CryptoScanner class — scan_tls, scan_ssh, scan_host, scan_network, generate_cbom; plus module-level functions scan_host, scan_directory, scan_network — inconsistent API), file_scanner.py (PEM file + SSH directory scanner), models.py (AssetFinding, ScanResult pydantic models), cli.py (typer-based CLI). Tests: test_scanner.py. The scanner uses ssl.create_default_context with check_hostname=False and verify_mode=ssl.CERT_NONE (scanner.py:77-78) — appropriate for a scanner but worth flagging.

planner/ — GNN migration planner microservice

FastAPI microservice (server.py) exposing /health, /plan, /plan/deadline. Loads model.pt at startup, falls back to a rule-based heuristic if torch is missing or the checkpoint is absent. Modules: model.py (v1 MigrationGNN — 3-layer GCN, this is the one actually used), model_v2.py (hybrid GCN+GATv2+centrality — DEAD CODE despite patent claims), model_legacy.py (duplicate of v1 — also dead code), train.py (ListMLE training script), predict.py (CLI + cbom_to_graph + predict_detailed), benchmark.py (multi-seed benchmark runner), data_generator.py (synthetic graph generator). requirements.txt: torch>=2.2, torch-geometric>=2.5, scipy, fastapi, uvicorn, pydantic, numpy. Dockerfile is python:3.11-slim based.

Other top-level directories

pilot/ contains run_pilot.py — an end-to-end First National Bank demo that chains scan → register → quantum analysis → plan → on-chain attest/migrate/audit → verify. notebooks/ contains 01_quantum_threat_demo.ipynb (Shor's algorithm resource estimate) and 08_bank_pilot.ipynb. scripts/ contains generate_abis.py (regenerates abis.ts and contracts.py from Forge artifacts — requires `forge build` first) and verify_all.sh (one-command full-stack verification script). docs/ contains 11 PHASE_*.md files (one per development phase), QTrust_Implementation_Guide.md (large reference), demo_run_of_show.md, playbook_review.md, PROMPT.md (the original Qwen 2.5 master build prompt — reveals the AI-generated origin), PATENT/ (draft_claims.md, filing_checklist.md, invention_disclosure.md, prior_art_survey.md), and assessments/ (three long-form assessments including two prior comprehensive audits). data/ contains cbom_for_planner.json (sample CBOM). docker-compose.yml ties api + webhook + postgres + planner + redis into a single compose stack.

3. Architecture & Design

System architecture and component boundaries

Q-Trust is layered across four tiers: (1) On-chain (Base L2) — Solidity registries behind TransparentUpgradeableProxy + UUPS, governed by TimelockController with a 2-day delay and a QTrustGovernance facade; (2) Off-chain services — Fastify API + viem relayer, Postgres indexer read-model, Redis (for claimed-but-unused webhook subscriptions), GNN planner microservice; (3) Clients — Next.js frontend, Python SDK, CLI inspector; (4) External — IPFS via Pinata, Basescan (optional), public IPFS gateways. Component boundaries are mostly clean: contracts never call external services, the backend talks to contracts only through viem, the SDK talks to contracts only through web3.py, the frontend never signs transactions directly (gasless flow is mediated by the relayer).

Data and control flow

The happy-path end-to-end flow is: scanner produces a CBOM JSON → SDK computes its SHA-256 → SDK calls AssetRegistry.registerCBOM (or signs an EIP-712 CBOMRegistration and the relayer submits via registerCBOMSigned) → the indexer catches the CBOMRegistered event and upserts into the Postgres assets table → frontend fetches /v1/assets/{id} → backend returns the indexed row (or falls back to direct RPC). Vendor attestation flow: vendor fetches their on-chain nonce via /v1/relay/nonce/{did} → signs EIP-712 ProductAttestation with their wallet → POSTs to /v1/relay/attestation → relayer recovers the signer, verifies the nonce matches the on-chain nonce, submits via attestProductSigned → contract verifies the signature again, increments the nonce, emits ProductAttested → indexer upserts into the attestations table. Migration recording flow is analogous. Audit flow: auditor (with AUDITOR_ROLE) calls AuditRegistry.postAudit with orgDid, result enum, counts, reportHash, reportURI; the contract enforces assetsMigrated ≤ migrationRegistry.getMigrationsByOrg(orgDid).length.

Major modules, classes, functions, and interfaces

Solidity: AssetRegistry (registerCBOM, registerCBOMSigned, updateCBOM, retireAsset, verifyAsset, getAssetsByOrg, getAllAssetIds), VendorRegistry (registerVendor, attestProduct, attestProductSigned, revokeAttestation, checkProductSupport, getAttestationsByVendor), MigrationRegistry (recordMigration, recordMigrationSigned, verifyMigration, getMigrationsByOrg), AuditRegistry (postAudit, getLatestAudit, getAuditsByOrg), QTrustGovernance (scheduleDeactivateVendor, scheduleRetireAsset, scheduleGrantRole, schedulePause, scheduleUnpause, execute). Backend: requireApiKey, getAsset, verifyAsset, getAssetsByOrg, getVendorAttestations, checkProductSupport, getMigrationsByOrg, getMigrationProgress, getLatestAudit, getOrgSummary, registerCBOM, attestProduct, recordMigration, relaySignedAttestation, relaySignedCBOMRegistration, relaySignedMigration, fanOut (dead). SDK: QTrustClient, VCIssuer, VCPresenter, VCVerifier, DIDResolver, TrustEvaluator, PinataClient. Inspector: CryptoScanner, scan_host, scan_directory, scan_network. Planner: MigrationGNN, TrustEvaluator, _heuristic_priority, _heuristic_risk, cbom_to_graph, predict_detailed, train.

Design patterns and architectural decisions

Proxy + UUPS upgradeability with _authorizeUpgrade gated to DEFAULT_ADMIN_ROLE allows future contract fixes without migration. OpenZeppelin AccessControl gives role-based permissions (REGISTRAR_ROLE, VENDOR_ROLE, MIGRATOR_ROLE, AUDITOR_ROLE, plus per-contract admin roles). EIP-712 typed-data signing with on-chain nonce replay protection (the contract re-checks the nonce against storage). Registry-per-concern separation (one contract per type of record) keeps gas costs predictable and allows independent upgrades. Timelock governance with a 2-day delay and a QTrustGovernance facade means no single key can pause, retire, or grant roles without a public notice period. Idempotent indexer upserts (ON CONFLICT DO UPDATE) handle replays safely. Graceful

degradation: when Postgres is unavailable, the API falls back to direct RPC reads; when Redis is unavailable, webhook subscriptions are disabled but the API still serves. The relay is a single-key signer today — a known single point of failure that should be replaced with a multisig (Safe) + meta-tx framework before production.

Coupling, cohesion, separation of concerns, extensibility

Coupling: contracts are loosely coupled (only MigrationRegistry and AuditRegistry hold references to other registries, both via constructor injection). Backend-to-contract coupling is through auto-generated ABIs (regenerable via scripts/generate_abis.py) — good. Frontend-to-backend coupling is via REST only — good. SDK-to-contract coupling is also via auto-generated ABIs. Cohesion: each contract is highly cohesive (one concern per contract). Each backend service module is cohesive (verify.ts = reads, attestation.ts = writes + relay, indexer.ts = event ingestion + Postgres queries, webhook.ts = delivery). Extensibility: UUPS allows contract upgrades. Adding a new registry is straightforward (write the contract, deploy behind proxy, add to indexer EVENTS list, regenerate ABIs). Adding a new read endpoint is one function in verify.ts and one route in server.ts. But: QTrustGovernance.schedulePause and scheduleUnpause hardcode the AssetRegistry.pause signature regardless of registryIndex (QTrustGovernance.sol:74-86) — this is an extensibility bug. It works today only because all four registries inherit Pausable and have identical pause() signatures; it would silently encode the wrong call for any future contract with a different pause signature.

Q-Trust System Architecture — Component Map (commit ec67f64)

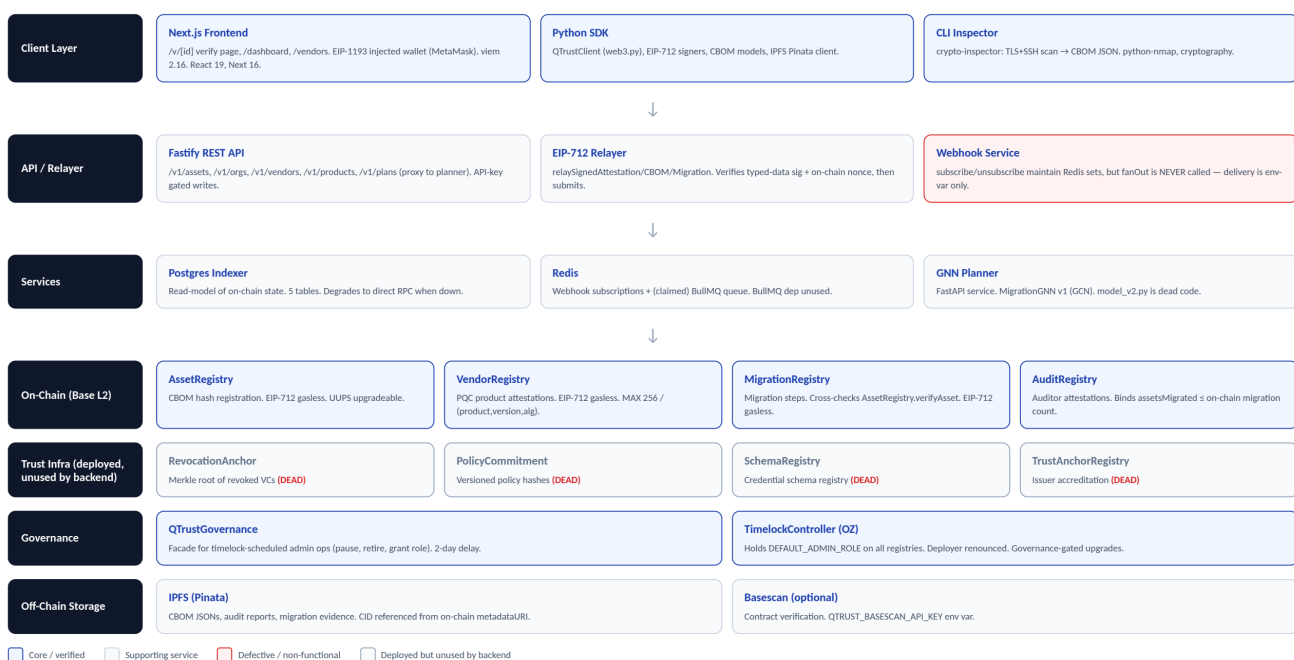


Figure 1. Q-Trust system architecture and component map. Navy-bordered nodes are core/verified. Red-bordered node (Webhook Service) is defective — fanOut is never called. Gray-bordered nodes (trust infra contracts) are deployed but unused by the backend.

4. Complete Code Walkthrough

4.1 Smart contracts

AssetRegistry (AssetRegistry.sol): the central CBOM registry. `_registerCBOM` (line 181) computes `assetId = keccak256(abi.encodePacked(orgDid, cbomHash, block.timestamp))`. This means two registrations of the same CBOM by the same org in the same second (same `block.timestamp`) would produce the same `assetId` and the second would revert with `AssetAlreadyExists` — correct behavior for replay protection, but worth noting that `block.timestamp` has 1-second resolution on Base, so two legitimate CBOM updates by the same org within the same second would collide. `updateCBOM` (line 210) allows the original `orgDid` or `DEFAULT_ADMIN` to update the hash. `retireAsset` (line 228) marks `active=false`. EIP-712 gasless path `registerCBOMSigned` (line 128) recovers the signer, checks `nonces[signer] == nonce`, increments the nonce, then calls `_registerCBOM` with the signer as `orgDid` — the SIGNER is recorded as the org, not the relayer. The domain separator is computed in `initialize()` (line 80) using `block.chainid` — note this is computed once at initialization; if the contract is deployed on a chain and then `chainId` changes (does not happen in practice), the separator would be stale. `Pausable` + `ReentrancyGuard` + `UUPS` all present and used.

VendorRegistry (VendorRegistry.sol): vendor registration is admin-only (`VENDOR_ADMIN_ROLE`); `attestProduct` requires `VENDOR_ROLE` and active vendor status. `_storeAttestation` (line 199) enforces `MAX_ATTESTATIONS_PER_PRODUCT = 256` (line 28) — a DoS protection that prevents a single (`productId`, `version`, `algorithm`) tuple from accumulating unbounded attestations. `attestProductSigned` (line 143) is the EIP-712 path; it requires the signer to be an already-registered, active vendor — so gasless attestations still require prior admin registration. `revokeAttestation` (line 308) requires attestation vendor or admin.

MigrationRegistry (MigrationRegistry.sol): the only contract with a cross-registry dependency. `initialize()` takes `assetRegistry_` (line 82) and stores it as a public immutable. `_recordMigration` (line 210) calls `assetRegistry.verifyAsset(assetId)` (line 222) and reverts with `AssetNotRegistered` or `AssetInactive` if the asset is missing or retired — this is the key integrity binding. It also rejects same-algorithm migrations (line 227) and empty `evidenceHash` (line 226). `verifyMigration` (line 255) is auditor-only and just sets `verified=true` — there is no on-chain verification of the evidence itself, just an attestation that an auditor approved it.

AuditRegistry (AuditRegistry.sol): `postAudit` (line 74) enforces three invariants: `reportHash` non-zero, `assetsMigrated ≤ assetsReviewed`, and `assetsMigrated ≤ on-chain migration count` (`migrationRegistry.getMigrationsByOrg(orgDid).length`). `auditId` is `keccak256(msg.sender, orgDid, reportHash, block.timestamp)` — collision-resistant per auditor per report per second. `getLatestAudit` (line 148) returns the last element of `_auditsByOrg[orgDid]` — `O(1)`.

RevocationAnchor, **PolicyCommitment**, **SchemaRegistry**, **TrustAnchorRegistry**: these four contracts are deployed by `Deploy.s.sol:99-153` behind `TransparentUpgradeableProxy`, with admin role handed to the timelock. They are well-designed (Merkle-root revocation, versioned policy hashes, schema registry with cross-domain equivalence, issuer accreditation with `validFrom/validUntil`). However, the backend has only stub endpoints for them (`server.ts:526-572` return hardcoded `0x00` hashes). The `VCVerifier` in the SDK does not actually query `RevocationAnchor`. These contracts are deployed but unused — either wire them up or remove them.

QTrustGovernance (QTrustGovernance.sol): a thin facade over `TimelockController`. Each `schedule*` function encodes the target call and calls `timelock.schedule(target, 0, data, bytes32(0), salt, DEFAULT_DELAY)` where `DEFAULT_DELAY = 2` days. The bug: `schedulePause` (line 74) and `scheduleUnpause` (line 82) always encode `AssetRegistry.pause` signature regardless of `registryIndex`, because they call `abi.encodeCall(AssetRegistry.pause, ())` with the `registryIndex` only used to pick the target address. This works because all four registries inherit `Pausable.pause()` with identical signature. It would silently produce the wrong call for any future registry with a different pause signature — an

extensibility landmine. The fix: define an IPausable interface and encode against that, or switch on registryIndex to encode each registry's own pause().

4.2 Backend (Fastify + viem)

server.ts: 656 lines. The route surface is broad — read routes (/v1/assets/:id, /v1/assets/:id/verify, /v1/orgs/:did/summary, /v1/orgs/:did/assets, /v1/orgs/:did/migrations, /v1/orgs/:did/audit, /v1/migrations/:id, /v1/vendors/:did/attestations, /v1/products/:id/support, /v1/plans/:did), write routes (/v1/write/assets, /v1/write/attestations, /v1/write/migrations — API-key gated), relay routes (/v1/relay/attestation, /v1/relay/cbom, /v1/relay/migration, /v1/relay/nonce/:did, /v1/relay/cbom-nonce/:did — EIP-712 gasless, no API key), evaluation routes (/v1/evaluate, /v1/credentials/issue, /v1/credentials/verify — STUBS), trust-infra query routes (/v1/revocation/:issuer, /v1/policies/:policyId/versions/:version, /v1/schemas/:schemaId, /v1/trust-anchors/:issuer — STUBS), webhook routes (/v1/webhooks/subscribe, /unsubscribe, /subscribers — Redis-backed but disconnected from delivery), and legacy aliases (/assets/:id, /migration/progress/:org).

requireApiKey (server.ts:97-107) has a critical dev hole: when QTRUST_API_KEYS is unset (line 99 returns 0-length array), the function calls done() and allows the request through. The intent ("No keys configured — dev mode, allow writes") is clear, but the production posture should be fail-closed: if NODE_ENV=production and API_KEYS is empty, refuse. The fix is two lines.

The /v1/evaluate endpoint (server.ts:406-473) implements a hardcoded policy with 5 clauses (no_rsa_1024 weight 0.3, tls_min_2048 weight 0.25, pqc_plan_exists weight 0.2, no_weak_hash weight 0.15, vendor_attestations weight 0.1). The confidence = satisfiedWeight / totalWeight, but totalWeight always sums to 1.0 (0.3+0.25+0.2+0.15+0.1), so confidence == satisfiedWeight — the division is misleading. The SDK has a proper TrustEvaluator (sdk/qtrust/trust.py) that accepts policy_clauses as a parameter — the backend should call that, not duplicate the logic.

The /v1/credentials/verify endpoint (server.ts:503-521) returns {valid: true, revoked: false} for ANY input — this is the most serious stub in the codebase because it actively misleads API consumers. The note says "Full VC verification with DID resolution is available via the Python SDK" — but the SDK's VCVerifier also does not verify signatures (see § 4.3).

attestation.ts: the relayer. The EIP712_DOMAIN constant (line 124) hardcodes chainId: 84532. config.ts supports QTRUST_USE_MAINNET=true which sets CHAIN.id = 8453, but the relayer's walletClient (line 34-38) is hardcoded to baseSepolia. So if someone deploys to mainnet Base, the relayer will sign and submit to baseSepolia — silent breakage. The relayer also re-creates its own publicClient (line 39-42) instead of importing from config.ts — duplicated configuration. relaySignedAttestation (line 152) is otherwise well-designed: recover typed-data address, read on-chain nonce, compare, submit, wait for receipt, find the ProductAttested log, extract attestationId. The waitForTransactionReceipt call (line 207) has no timeout — if the tx is stuck in the mempool, the request hangs indefinitely.

indexer.ts: the Postgres read-model. EVENTS array (line 35) lists 7 event specs across 4 contracts. applyLog (line 131) dispatches per spec.key and parameterizes the upsert. backfill (line 195) chunks by 2000 blocks (line 207) and advances the cursor after each chunk. watchLive (line 223) subscribes to live events. Two issues: (1) applyLog and setCursor are NOT in a transaction — if Postgres crashes between the upsert and the cursor advance, the next backfill re-applies the same log. The upserts are idempotent for INSERT/UPDATE, but the revoked toggle on AttestationRevoked could double-toggle if applied twice in sequence with a competing write. (2) watchLive's onLogs callback (line 229-235) has no try/catch — an exception in applyLog will crash the watcher silently. startIndexer (line 241) catches the backfill failure and logs a warning, but live watcher failures are unhandled.

webhook.ts: SSRF protection in isPublicHttpsUrl (line 22) is good but incomplete. It blocks localhost, 0.0.0.0, .local, .internal, 127/8, 10/8, 192.168/16, 172.16-31, ::1, fe80:. It MISSES: 169.254.0.0/16 (link-local), fc00::/7 (IPv6 ULA), ::ffff:127.0.0.1 (IPv4-mapped IPv6), and IPv6 site-local fec0:. fanOut (line 88) is exported but never imported by any other file (grep-confirmed across the entire repo). The webhook

subscribe/unsubscribe endpoints maintain Redis sets under subscribers:* keys, but nothing reads those sets — the actual delivery path is the QTRUST_WEBHOOKS env var (line 89). The Redis-backed subscribe system is non-functional theater.

4.3 Python SDK

client.py: QTrustClient. The constructor reads private_key, rpc_url, and 4 registry addresses from env vars with sensible anvil defaults. It instantiates web3, sets up ExtraDataToPOAMiddleware (appropriate for PoA chains like Base Sepolia), and creates 4 contract instances. The EIP-712 signers (sign_attestation, recover_attestation_signer) use encode_typed_data from eth-account — correct. BASE_SEPOLIA_CHAIN_ID = 84532 is hardcoded (line 23), so the SDK cannot sign for mainnet Base without code changes.

vc.py: the W3C Verifiable Credentials module. The VCIssuer.issue method (line 173) correctly builds a VerifiableCredential and signs it with Ed25519 via _ed25519_sign (line 113) using pynacl. The VCVerifier.verify_credential (line 319) is where the problems start. Line 326-340 is a try/except around DID resolution: if resolution fails, the except handler returns VCVerificationResult(valid=not expired and not revoked, ...) — but `revoked` is referenced before assignment in this scope. Python would raise NameError, defeating the try/except. Lines 344-354 handle the case where the proof is present: it builds the canonical message, extracts the signature, fetches the issuer's authentication key from the DID document — and then line 354 just appends "Signature verification delegated to holder" to the explanation_parts list. The actual Ed25519 verification (_ed25519_verify) is NEVER called. The function returns valid=true (since not expired and not revoked, both default to False) for any input with a proof. The revocation check (line 366-369) is also a stub — it just appends "Revocation check requires on-chain root verification" without querying the RevocationAnchor contract. The verify_credential_sync method (line 381) has the same bugs in mirror form.

did.py: DIDResolver for did:web. _did_to_url (line 41) correctly maps did:web:example.com → https://example.com/.well-known/did.json and did:web:example.com:path → https://example.com/path/did.json. The cache (5-min TTL, line 39) is reasonable. get_authentication_key (line 133) returns the first authentication method — but if the auth method is a string reference (e.g., "did:web:example.com#key-1"), it correctly resolves against verificationMethod. The code is functional but only supports did:web (no did:key, did:ethr, did:pkh).

trust.py: TrustAssessment + TrustEvaluator. The TrustEvaluator.evaluate method (line 99) takes subject_did, policy_id, policy_version, evidence list, and optional policy_clauses dict. For each clause, it checks if any evidence satisfies the required_claims (line 132-135), accumulates weights, and computes confidence = satisfied_weight / total_weight. The policy is deterministic — same inputs produce the same output. The compute_hash method (line 58) produces a canonical SHA-256 for reproducibility. This module is well-designed and tested (test_trust.py). The backend's /v1/evaluate endpoint should use this module instead of reimplementing it inline.

4.4 Inspector (crypto scanner)

scanner.py: CryptoScanner.scan_tls (line 66) connects with ssl.create_default_context and CERT_NONE — appropriate for a scanner. It extracts certificate metadata (issuer, subject, serial, fingerprint, validity, cipher, signature algorithm). scan_ssh (line 152) reads the SSH banner and parses the kex_init packet to extract offered host key algorithms, then calls _ssh_keys_scan (line 228) which shells out to ssh-keyscan. _ssh_keys_scan parses the ssh-rsa key blob: the comment at line 279 says "ssh-ss" — typo, should be "ssh-rsa" (the actual code parses correctly despite the comment). _ssh_to_asset (line 452) hardcodes vendor = "openssh" (line 466) — three s's, a typo that would appear in the generated CBOM. The scan_tls method has a structural quirk: after the return AssetFinding(...) at line 131, there is a comment block (# SSH scanning) at line 149 indented inside the method — Python allows trailing comments but the indentation is misleading. The module exposes two scan_host functions: CryptoScanner.scan_host (line

315, returns dict) and module-level `scan_host` (line 519, returns `ScanResult`) — inconsistent API that the pilot imports as ``from qtrust_inspector.scanner import scan_host``, getting the module-level one.

`file_scanner.py`: `scan_pem_files` (line 12) globs for `*.pem`, `*.crt`, `*.key`, `*.cert`. For each file, it tries `x509.load_der_x509_certificate(content)` — but a `.pem` file contains PEM-armored base64, not raw DER. The cryptography library provides `load_pem_x509_certificate` for this. The current code will fail to parse any PEM file (the inner `try/except` at line 23-35 catches the exception and falls through to `load_pem_private_key`, which will also fail because a `.pem` containing a certificate is not a private key). The function silently produces no findings for PEM files. This is a real bug — the file scanner is non-functional for the common case.

4.5 Planner (GNN microservice)

`model.py`: MigrationGNN v1 — 3-layer GCN with residual connections and two prediction heads (order, risk). This is the model actually used by `train.py` (line 123), `predict.py` (line 157, 204), and `server.py` (line 59). Input features are 6-dimensional: `algorithm_type`, `key_size`, `vendor_pqc_ready`, `criticality`, `days_to_deadline`, `required_rate`. Forward pass (line 101) does 3 GCN layers with ReLU and residual connections, then per-node Linear heads for order and risk logits.

`model_v2.py`: MigrationGNN v2 — 175-line "patent-grade hybrid": `LayerNorm + GCN → GATv2Conv → GCN` with `BatchNorm + Dropout + Residual`, plus centrality augmentation (in/out degree from `edge_index`) and 2-layer MLP heads. The patent materials (`docs/PATENT/invention_disclosure.md`, `docs/assessments/QTrust_Comprehensive_Assessment_Post_P0.md`, `docs/assessments/QTrust_2030_Blueprint.md`) all reference `model_v2.py` as the production model. But `train.py`, `predict.py`, and `server.py` all import from `.model (v1)`, never from `.model_v2`. The v2 model is dead code. If a developer attempts to load a checkpoint trained with v2 architecture, they would get a `state_dict` mismatch — and the `predict.py` fallback (line 159-162) silently uses `strict=False`, so the model would load with random weights for the new layers. This is a silent correctness bug waiting to happen.

`server.py`: FastAPI service. `_load_model` (line 43) is called both at startup (line 145, `@app.on_event("startup")` — deprecated in FastAPI, should be `lifespan` context) and inside the `/plan` endpoint (line 158, "if not `_model_info`: `_load_model()`"). This is racy: if a request arrives during startup, both calls might race. The `/plan` endpoint (line 155) tries `cbom_to_graph` first, falls back to manual parsing on `ImportError` (`torch-geometric` missing). The fallback path skips the dependency graph entirely and uses `asset_records` directly without edges — the GNN would see no edges and the prediction would be feature-only. The `_build_schedule` function (line 241) greedily backfills from the deadline, which is reasonable for the demo but not optimal for parallelizable migrations.

Dockerfile (`planner/Dockerfile:10`): `COPY model.pt model.pt` — but `.gitignore:28` excludes `planner/model.pt`. So a fresh clone has no `model.pt`, and ``docker build`` would succeed (`COPY` fails silently if the source is missing? Actually it fails the build) — wait, actually `COPY` with a missing source will fail the build. So a fresh clone CANNOT build the planner image. The `model.pt` in the repo at `planner/model.pt` is currently tracked (I verified it exists in the file listing) — so it was force-added at some point, but the `.gitignore` rule means subsequent re-trains would not commit a new model. This is inconsistent.

4.6 Pilot (run_pilot.py)

`pilot/run_pilot.py`: 252-line end-to-end First National Bank demo. `step1_scan` (line 49) calls `scan_host` and falls back to a synthetic CBOM if no findings. Line 78: ``return {"cbom": cbom, "synthetic": True}`` — but the variable ``cbom`` is only constructed in the `else` branch at line 80-94. On the no-findings path, line 57-77 builds ``result`` (a CBOM), not ``cbom``. So line 78 references an undefined ``cbom`` — a `NameError` that would crash the pilot on the no-findings path. Since `example.com` (the default target) usually does return TLS findings, this path is rarely hit in practice, but it is a real bug.

4.7 End-to-end execution paths

Path 1 — Register CBOM via SDK: QTrustClient.register_cbom(cbom) → web3.py contract call → AssetRegistry.registerCBOM → CBOMRegistered event → indexer.watchLive → applyLog("assets", log) → Postgres upsert → /v1/assets/:id returns the indexed row. Verified by sdk/tests/e2e_anvil.py:72-89.

Path 2 — Gasless vendor attestation: vendor fetches nonce via /v1/relay/nonce/:did → SDK signs EIP-712 ProductAttestation → POST /v1/relay/attestation → relaySignedAttestation recovers signer, checks on-chain nonce, submits attestProductSigned → contract verifies sig + nonce, increments nonce, emits ProductAttested → indexer applies. Verified by sdk/tests/e2e_anvil.py:273-307.

Path 3 — Public verification: GET /v/:id on the frontend → fetch API → backend /v1/assets/:id/verify → verify.ts:verifyAsset → assetRegistry.read.verifyAsset → returns (exists, active, orgDid). The frontend renders the result. No auth, no rate limit beyond the global 120/min. Verified by frontend/src/app/v/[id]/page.tsx.

4.8 Dead, duplicated, unreachable, placeholder, and suspicious code

- DEAD: webhook.ts:fanOut (line 88) — exported, never imported anywhere. The webhook subscribe/unsubscribe endpoints maintain Redis sets that nothing reads.
- DEAD: planner/qtrust_planner/model_v2.py — 175 lines, never imported by train.py, predict.py, or server.py. Referenced only in patent docs.
- DEAD: planner/qtrust_planner/model_legacy.py — duplicate of model.py (identical class definition). Neither file is used by the v2 import path.
- DEAD: backend/package.json:17 "bullmq" — listed as a dependency but never imported in any backend source file. README claims "BullMQ webhooks" — inaccurate.
- DEAD: frontend/.env.example:11 "NEXT_PUBLIC_DYNAMIC_ENV_ID" — Dynamic Labs SDK was removed per the post-P0 assessment; env var remains.
- STUB: server.ts:503-521 /v1/credentials/verify — returns {valid: true, revoked: false} for ANY input.
- STUB: server.ts:526-560 /v1/revocation, /v1/policies, /v1/schemas, /v1/trust-anchors — all return hardcoded 0x00 hashes.
- STUB: sdk/qtrust/vc.py:354 — "Signature verification delegated to holder" — VCVerifier does not verify Ed25519 signatures.
- STUB: sdk/qtrust/vc.py:369 — "Revocation check requires on-chain root verification" — RevocationAnchor contract is never queried.
- BUG: pilot/run_pilot.py:78 — references `cbom` before assignment on the no-findings path (NameError).
- BUG: sdk/qtrust/vc.py:333-340 — `revoked` referenced before assignment in DID resolution except handler (NameError).
- BUG: inspector/file_scanner.py:25 — uses load_der_x509_certificate on PEM files (wrong loader).
- BUG: backend/src/services/attestation.ts:127,233,334 — EIP-712 chainId hardcoded to 84532; breaks mainnet deployment.
- BUG: backend/src/services/attestation.ts:36 — walletClient chain hardcoded to baseSepolia; breaks mainnet.
- BUG: backend/src/lib/config.ts vs frontend/src/lib/config.ts — frontend uses QTRUST_REGISTRY_ADDRESS (singular, likely typo) for assetRegistry at line 31; backend uses QTRUST_ASSET_REGISTRY_ADDRESS. Inconsistent env var names.

- TYPO: inspector/qtrust_inspector/scanner.py:466 — vendor = "opensssh" (three s's).
- TYPO: inspector/qtrust_inspector/scanner.py:279 — comment "ssh-ss" should be "ssh-rsa".
- SUSPICIOUS: QTrustGovernance.schedulePause/Unpause (QTrustGovernance.sol:74-86) — hardcode AssetRegistry.pause signature regardless of registryIndex.

5. Dependencies & Runtime

Languages, frameworks, libraries, versions, tooling

Languages: Solidity 0.8.24 (foundry.toml:5; QTrustGovernance.sol uses pragma ^0.8.24 — slightly looser, but consistent), TypeScript ES2022 with NodeNext module resolution and strict mode (backend/tsconfig.json), Python >=3.10 (sdk/pyproject.toml:10, inspector/pyproject.toml:11), JavaScript ESM (frontend, Next.js 16).

Frameworks: Foundry (forge compiler + test runner, version locked in foundry.lock), Fastify v5 (^5.12.1, backend), Next.js 16 (frontend, very new — released October 2024, still has rough edges with React 19), React 19 (^19.0.0, frontend), FastAPI (planner, version unfrozen in requirements.txt), PyTorch Geometric (planner, torch>=2.2, torch-geometric>=2.5).

Key libraries: viem ^2.16.0 (backend + frontend — EVM client), web3>=6.0.0 (SDK — EVM client), eth-account>=0.10.0 (SDK — EIP-712 signing), pydantic>=2.0.0 (SDK, inspector — models), @openzeppelin/contracts (submodule, version locked in foundry.lock), @fastify/cors ^9.0.1, @fastify/rate-limit ^11.2.0, bullmq ^5.12.0 (UNUSED), dotenv ^16.4.5, ioredis ^5.4.1, pg ^8.23.0, @tanstack/react-query ^5.51.0 (frontend), reactflow ^11.11.4 (frontend, deprecated — should be @xyflow/react), cryptography>=41.0.0 (inspector), python-nmap>=0.7.1 (inspector), typer>=0.9.0 + rich>=13.0.0 (inspector CLI), scipy>=1.12 (planner, for kendalltau), httpx (SDK, for DID resolution), requests (SDK, for Pinata), pynacl (SDK optional, for Ed25519 — NOT in pyproject.toml dependencies, must be installed separately).

Dependency relationships and risks

Backend → viem → Base L2 RPC. Backend → pg → Postgres. Backend → ioredis → Redis. Backend → fetch → planner. Frontend → viem → Base L2 RPC (direct read, no backend). Frontend → fetch → backend. Frontend → fetch → IPFS gateway. SDK → web3 → Base L2 RPC. SDK → eth-account (signing). SDK → httpx → DID URLs. SDK → requests → Pinata. Inspector → cryptography + ssl + socket + subprocess(ssh-keyscan). Planner → torch + torch-geometric. All clients → Base Sepolia RPC by default (QTRUST_BASE_SEPOLIA_RPC).

Risks: (1) Next.js 16 + React 19 are very new; breaking changes between minor versions are likely. (2) reactflow ^11.11.4 is deprecated; the maintainers renamed the package to @xyflow/react for v12. (3) bullmq is an unused dependency that increases the install footprint and attack surface for no benefit. (4) pynacl is not in the SDK's pyproject.toml dependencies, so a fresh `pip install -e ./sdk` cannot do Ed25519 — the VCIssuer._sign method raises ImportError. (5) The backend Dockerfile uses oven/bun:1.1 as the base image, but the build step uses `bun run build` which calls tsc — this works but means the production runtime is bun, not Node. The README says `npm install` — this works on bun too, but it is a mismatch in documented vs. actual tooling. (6) The planner has no version pins for fastapi, uvicorn, scipy, numpy — a fresh install could pull in a breaking major version. (7) The OpenZeppelin contracts submodule is pinned in foundry.lock but the submodule itself tracks a tag/branch — reproducibility depends on the lock file. (8) viem 2.16 is recent but the EIP-712 recoverTypedDataAddress API has had breaking changes between 2.x minor versions in the past.

Build/run requirements and environment assumptions

Build requirements: Foundry (forge), Node.js 18+ or bun 1.1+, Python 3.10+. For contracts: `forge build` then `forge test`. For backend: `npm install` then `npm run build` (or `bun install` + `bun run build` per the Dockerfile). For frontend: `npm install` then `npm run build`. For SDK: `pip install -e ./sdk`. For inspector: `pip install -e ./inspector`. For planner: `pip install -r planner/requirements.txt`. For docker-compose: Docker 24+ with compose v2. Environment assumptions: the contracts expect Base

Sepolia (chainId 84532); the backend expects anvil or a Base Sepolia RPC at <http://127.0.0.1:8545>; the planner expects model.pt to exist at planner/model.pt; the indexer expects Postgres at postgres://qtrust:qtrust@postgres:5432/qtrust (docker-compose) or postgres://localhost:5432/qtrust (local).

6. Configuration & Environment

Configuration mechanisms

All configuration is via environment variables, loaded with dotenv at module load time. backend/src/config.ts reads 14 env vars; backend/src/services/attestation.ts reads 4 more (QTRUST_RELAYER_PRIVATE_KEY, QTRUST_ASSET/VENDOR/MIGRATION_REGISTRY_ADDRESS); frontend/src/lib/config.ts reads QTRUST_BASE_SEPOLIA_RPC, QTRUST_USE_MAINNET, QTRUST_REGISTRY_ADDRESS (typo), QTRUST_VENDOR_REGISTRY_ADDRESS, QTRUST_MIGRATION_REGISTRY_ADDRESS, QTRUST_AUDIT_REGISTRY_ADDRESS, QTRUST MOCK_WALLET; planner/server.py reads QTRUST_MODEL_PATH and QTRUST_DEADLINES_PATH; SDK reads QTRUST_DEPLOYER_PRIVATE_KEY, QTRUST_BASE_SEPOLIA_RPC, and the 4 registry addresses; inspector does not read env vars (CLI args only). docker-compose.yml hardcodes the service-level env vars (NODE_ENV, PORT, HOST, QTRUST_REDIS_URL, QTRUST_PG_URL, QTRUST_PLANNER_URL, QTRUST_BASE_SEPOLIA_RPC=http://host.docker.internal:8545) and uses env_file: .env for the rest.

Environment variables and secrets

The full env-var surface is documented in README.md:81-94 and backend/.env.example. The variables fall into four categories: (1) Network — QTRUST_BASE_SEPOLIA_RPC, QTRUST_USE_MAINNET; (2) Identity — QTRUST_DEPLOYER_PRIVATE_KEY, QTRUST_RELAYER_PRIVATE_KEY, QTRUST_BASESCAN_API_KEY; (3) Addresses — QTRUST_ASSET/VENDOR/MIGRATION/AUDIT_REGISTRY_ADDRESS; (4) Services — QTRUST_REDIS_URL, QTRUST_PG_URL, QTRUST_PLANNER_URL, QTRUST_WEBHOOKS, QTRUST_CORS_ORIGINS, QTRUST_API_KEYS, QTRUST_INDEXER_FROM_BLOCK; (5) Frontend — NEXT_PUBLIC_QTRUST_API_URL, NEXT_PUBLIC_IPFS_GATEWAY, NEXT_PUBLIC_DYNAMIC_ENV_ID (dead), QTRUST MOCK_WALLET. No secrets manager integration (no AWS Secrets Manager, no Vault, no Doppler). All secrets live in .env files committed to git history if not careful.

Dev/prod differences

docker-compose.yml sets NODE_ENV=production for both api and webhook containers. The .env file is shared between local dev and docker-compose. There is no separate .env.production or .env.staging. The CORS_ORIGINS env var defaults to "*" (config.ts:45-48), which Fastify interprets as "allow all origins" (server.ts:86). In production this should be set to a specific origin list. The API_KEYS env var defaults to empty (config.ts:51-54), which (per § 4.2) opens the write routes — a production hazard.

Hard-coded values and environment-specific assumptions

Hardcoded: chainId 84532 in three EIP712_DOMAIN constants in attestation.ts (lines 127, 233, 334) — these should be CHAIN.id or read from the contract. baseSepolia chain object hardcoded in attestation.ts:36-38 for the relayer's walletClient and publicClient — should import from config.ts. BASE_SEPOLIA_CHAIN_ID = 84532 in sdk/qtrust/client.py:23 — should be a parameter. QTrustGovernance.DEFAULT_DELAY = 2 days (QTrustGovernance.sol:27) — reasonable but not configurable. VendorRegistry.MAX_ATTESTATIONS_PER_PRODUCT = 256 (VendorRegistry.sol:28) — DoS cap, reasonable. INDEXER_FROM_BLOCK = 0 by default (config.ts:64) — should be set to the contract deployment block in production. CORS_ORIGINS defaults to "*" — fine for dev, dangerous for prod. Mock wallet address 0xf39Fd6e51aad88F6F4ce6aB8827279cFfFb92266 (frontend/src/lib/config.ts:77) — the well-known anvil Account #0 address, used as a dev fallback.

Committed secrets

The anvil dev private key 0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80 appears in 6 committed files: backend/.env.example:12, pilot/run_pilot.py:43, sdk/tests/e2e_anvil.py:20, sdk/tests/test_client.py (implied by grep), sdk/tests/run_e2e.sh:22, scripts/verify_all.sh:47. This is the well-known Hardhat/Foundry anvil dev key (Account #0) — it does not control real funds on any mainnet. Its presence in test files (e2e_anvil.py, run_e2e.sh, verify_all.sh) is acceptable. Its presence in pilot/run_pilot.py is borderline (the pilot is a demo, but it sets the env var via os.environ.setdefault which is fine). Its presence in backend/.env.example is the worst pattern — .env.example is a template that users copy to .env; if a user copies it verbatim and runs in production with the anvil key, the relayer would sign on-chain transactions with a key anyone can derive. The key is not technically a "leaked secret" but it is a pattern anti-recommendation: .env.example should use a placeholder like 0xYOUR_PRIVATE_KEY_HERE.

7. API / Interfaces / Integrations

Internal and external APIs

Internal API: the Fastify REST API at `/v1/*` (server.ts). Full route inventory: 11 read routes, 3 admin-write routes (API-key gated), 5 relay routes (EIP-712 gasless, no API key), 3 evaluation routes (stubs), 4 trust-infra query routes (stubs), 3 webhook routes (Redis-backed but disconnected from delivery), 2 legacy alias routes (`/assets/:id`, `/migration/progress/:org`), 1 health route. Frontend API: GET `/api/vendor-registry-address` (exposes the VendorRegistry proxy address for browser EIP-712 signing) plus the Next.js rewrite rule that proxies `/api/:path*` to the Fastify backend.

External integrations: (1) Base L2 RPC (default `http://127.0.0.1:8545` for local anvil, or `https://sepolia.base.org` for testnet, or a custom RPC URL via env var). (2) Pinata IPFS API (`sdk/qtrust/ipfs.py:13`, `https://api.pinata.cloud/pinning/`) — uses `pinata_api_key` + `pinata_secret_api_key` headers. (3) Public IPFS gateway (frontend, default `https://ipfs.io/ipfs/` via `NEXT_PUBLIC_IPFS_GATEWAY`). (4) Basescan API for contract verification (`QTRUST_BASESCAN_API_KEY`, used by ``forge verify-contract``). (5) DID resolution (SDK `DIDResolver`) fetches `https://<domain>/.well-known/did.json` for `did:web` DIDs. (6) External webhook URLs (any HTTPS URL the subscriber registers).

Request/response and message flows

CBOM registration flow: POST `/v1/write/assets` with `{cbomHash: 0x..., metadataURI: ipfs://...}` and `x-api-key` header → `requireApiKey` → `registerCBOM` → `walletClient.writeContract` → `txHash` → `{txHash, relay}`. Gasless CBOM flow: GET `/v1/relay/cbom-nonce/:did` → `{did, nonce}` → client signs EIP-712 `CBOMRegistration` → POST `/v1/relay/cbom` with `{cbomHash, metadataURI, nonce, signature}` → `relaySignedCBOMRegistration` recovers signer, checks nonce, submits → `{txHash, orgDid, assetId}`. Attestation flow: similar via `/v1/relay/attestation`. Migration flow: similar via `/v1/relay/migration`. Verification flow: GET `/v1/assets/:id/verify` → `verifyAsset` → `assetRegistry.read.verifyAsset` → `{asset_id, exists, active, org_did, chain_id, chain_name, verified_at}`.

Authentication and authorization mechanisms

Three auth mechanisms: (1) API key (`x-api-key` header) for admin write routes (`/v1/write/assets`, `/v1/write/attestations`, `/v1/write/migrations`). The key is compared against `QTRUST_API_KEYS` (a comma-separated list, `config.ts:51-54`). Comparison is via `API_KEYS.includes(key)` — not constant-time, so timing attacks are theoretically possible (low risk in practice given network jitter). When `QTRUST_API_KEYS` is empty, all writes are allowed (`server.ts:99-102`) — the dev hole. (2) EIP-712 signature for relay routes — the signer is recovered from the typed-data signature, the on-chain nonce must match, and the contract re-verifies the signature before recording. This is the gasless flow and is properly designed. (3) On-chain role-based access (`REGISTRAR_ROLE`, `VENDOR_ROLE`, `MIGRATOR_ROLE`, `AUDITOR_ROLE`, plus per-contract admin roles) enforced by OpenZeppelin `AccessControl` in the contracts. Direct contract calls bypass the relay and require the caller to hold the appropriate role.

Error handling and integration failure modes

Most routes have try/catch that returns JSON errors with appropriate status codes (400 for bad input, 422 for contract reverts, 503 for service unavailable, 404 for not found, 401 for bad API key). However, several read routes (`server.ts:121-125`, `136-143`, `632-640`) lack try/catch and will return a 500 with a raw error message on contract revert. Contract revert messages are surfaced unmodified — strings like "execution reverted" with no ABI-decoded error selector. The relayer's `relaySignedAttestation` catches signature recovery failures and rethrows with a human-readable message ("EIP-712 signature verification failed:

invalid signature"). The planner proxy (server.ts:209-225) has a 60-second timeout and returns 503 if the planner is unavailable. The Redis connection has lazyConnect with maxRetriesPerRequest: 1 (server.ts:73) — if Redis is down, the webhook subscribe endpoint returns 503 but the rest of the API works. The Postgres pool has no reconnect logic — if Postgres goes down mid-request, the pool's acquire will throw and the route will 500.

8. Database & Data Layer

Schema and models

backend/src/db/schema.sql defines 5 tables: assets (asset_id TEXT PK, org_id, cbom_hash, metadata_uri, timestamp BIGINT, last_updated BIGINT, active BOOL, tx_hash, block_number BIGINT, first_seen TIMESTAMPTZ), attestations (attestation_id TEXT PK, vendor_id, product_id, version, algorithm, supported BOOL, evidence_uri, timestamp BIGINT, revoked BOOL, tx_hash, block_number, first_seen), migrations (migration_id TEXT PK, asset_id, org_id, from_algorithm, to_algorithm, evidence_hash, evidence_uri, timestamp, verified BOOL, tx_hash, block_number, first_seen), audits (audit_id TEXT PK, org_id, auditor_id, result INT, assets_reviewed BIGINT, assets_migrated BIGINT, report_hash, report_uri, timestamp, tx_hash, block_number, first_seen), and indexer_state (key TEXT PK, block BIGINT, updated_at TIMESTAMPTZ). Indexes on org_id, vendor_id, (product_id, version, algorithm), asset_id. All PKs are TEXT (the on-chain IDs are bytes32 hex strings). The schema is read-model only — the blockchain is the source of truth.

Queries and persistence logic

indexer.ts:applyLog (line 131) dispatches per event spec.key and uses INSERT ... ON CONFLICT (asset_id/attestation_id/migration_id/audit_id) DO UPDATE — idempotent replays. The upserts are parameterized (\$1, \$2, ...) — no SQL injection. queryAssets, queryAttestations, queryMigrations, querySummary (lines 260-338) are parameterized paginated SELECTs with COUNT(*) for total. The LIMIT/OFFSET parameters are interpolated as \$N positionally based on whether org/vendor filter is present — this is correct but the position-based parameter numbering is fragile (a refactor that adds a WHERE clause could misnumber the params).

Migrations

There is no schema migration tool. schema.sql is mounted into the Postgres docker container's /docker-entrypoint-initdb.d/ directory (docker-compose.yml:75), which means it runs only on first boot. Any subsequent schema change requires manual SQL: either a new ALTER TABLE in a migration script (not present) or destroying the volume and re-initializing. There is no Prisma, no alembic, no knex, no golang-migrate. For a project at this stage this is acceptable but will become a problem once the schema stabilizes and changes need to be tracked.

Data validation and integrity

On-chain integrity is enforced by the contracts (AssetRegistry.verifyAsset, MigrationRegistry cross-checks, AuditRegistry count binding). Off-chain integrity is enforced by the indexer's idempotent upserts and by the schema's NOT NULL constraints on key fields. There is no application-level validation of CBOM contents (the SDK's schema.py defines Pydantic models, but the backend accepts any cbomHash as long as it starts with "0x" — server.ts:250). The parseAssetId function (config.ts:67-75) validates 66-char hex format. Pagination offset/limit is coerced via Number() (server.ts:148-149) — Number("abc") is NaN, and Math.max(0, NaN) is NaN, which would then be passed to the SQL query as NaN — Postgres would reject it, but the error would be a 500 not a 400.

Transactions and concurrency concerns

The indexer's applyLog and setCursor are NOT wrapped in a transaction (indexer.ts:131-177 and 185-192). If Postgres crashes between the upsert and the cursor advance, the next backfill re-applies the same log. For INSERT/UPDATE events this is idempotent (ON CONFLICT DO UPDATE produces the same row). For AttestationRevoked, the upsert sets revoked=TRUE; if applied twice, it sets revoked=TRUE again —

idempotent. But if a migration is recorded and then an auditor marks it verified, and the indexer replays both events out of order due to a cursor gap, the final state is still correct because verified is just a bool toggle. The real risk is the watchLive callback (line 229-235): if applyLog throws, the watcher's onLogs callback throws, and the cursor is never advanced — subsequent events would be applied but the cursor would be stuck at the failed block. There is no retry, no DLQ, no alerting.

Potential data-loss or consistency risks

Risk 1: The indexer_state cursor can advance past a not-yet-applied log if watchEvent onLogs fires for a block N+1 before applyLog(block N) completes. This is unlikely (watchEvent is sequential per event) but the code does not guard against it. Risk 2: A reorg on Base (uncommon on L2 but possible) would invalidate already-applied logs — the indexer has no reorg handling (no block hash tracking, no rollback). Risk 3: The Postgres Pool max=10 (indexer.ts:25) can be exhausted under high event burst — applyLog calls would queue and the watchLive callback would block, potentially missing events (viem's watchEvent has an internal queue, but its size is implementation-defined). Risk 4: The indexer degrades to direct RPC reads when Postgres is down, but the direct RPC reads (e.g., getMigrationProgress in verify.ts:319-341) do N+1 queries (one for the list, one per item) — slow under load.

9. Security Audit

Authentication and authorization

Smart contracts: OpenZeppelin AccessControl with REGISTRAR_ROLE, VENDOR_ROLE, MIGRATOR_ROLE, AUDITOR_ROLE, plus per-contract admin roles (VENDOR_ADMIN_ROLE, ISSUER_ADMIN_ROLE, POLICY_AUTHORITY_ROLE, SCHEMA_AUTHORITY_ROLE, GOVERNANCE_ROLE). All write functions are role-gated. The Deploy script grants REGISTRAR/MIGRATOR/AUDITOR roles to the deployer at construction (Deploy.s.sol:78, 88-89, 30) and then hands DEFAULT_ADMIN_ROLE to the timelock (Deploy.s.sol:171-180). The deployer keeps the operational roles forever (they are not renounced) — this means the deployer can still register CBOMs, record migrations, and post audits directly, bypassing the timelock. This is a design choice (the docs note it for the pilot) but it is a privilege escalation risk if the deployer key is compromised.

Backend: API key (x-api-key) for /v1/write/* routes. When QTRUST_API_KEYS is unset, the dev hole (server.ts:99-102) allows all writes — this is the most serious backend auth issue. The fix is fail-closed: if NODE_ENV=production and API_KEYS is empty, refuse. The relay routes (/v1/relay/*) do not require an API key — they verify the EIP-712 signature instead. This is correct for the gasless flow, but it means anyone can submit a signed attestation to the relayer — the relayer pays the gas. There is no rate limit per-vendor, no spam filter. A motivated attacker could submit a flood of invalid signatures (each rejected by recoverTypedDataAddress) and waste relayer CPU.

Input validation and sanitization

Backend: ad-hoc body shape checks (e.g., server.ts:250 ``if (!body.cbomHash || !body.cbomHash.startsWith("0x"))``). No JSON schema validator is registered (no ajv, no zod, no @fastify/type-provider). parseAssetId (config.ts:67-75) enforces 66-char hex format for asset IDs. Pagination offset/limit is coerced via Number() with NaN risk. Vendor/product IDs are arbitrary strings — no length cap, so a caller could submit a 1MB productId and the contract would store it (subject to gas limits). The productId/version/algorithm are passed directly to the contract via writeContract — no client-side sanitization. metadataURI and evidenceURI are arbitrary strings (the contract stores them as-is) — no URL validation, no scheme allowlist (could be javascript:, file://, etc. — though the contract just stores the string, the frontend would render it).

Injection risks

SQL injection: all Postgres queries use parameterized \$1/\$2/... placeholders (indexer.ts, verify.ts). No string concatenation in queries. Low risk. Command injection: inspector/scanner.py:234-239 uses subprocess.run(["ssh-keyscan", "-p", str(port), "-T", str(DEFAULT_TIMEOUT), host], ...) — list args (not shell=True), so shell metacharacters in host are not interpreted. However, the ssh-keyscan binary itself may interpret some arguments (e.g., a host starting with "-" could be parsed as a flag). The fix: pass "--" before the host argument. XSS: Next.js React escapes by default; no dangerouslySetInnerHTML seen in any component. The IPFS metadata fetched by fetchIpfsJson (api.ts:236) is parsed as JSON and rendered — if the JSON contains script tags, React would escape them. Low risk.

Secrets and credential exposure

No real secrets in the repository. The anvil dev key (0xac09...ff80) is in 6 files but is the well-known Account #0 test key — not real funds. No AWS keys, no Pinata keys, no production private keys. The .gitignore properly excludes .env files (with !.env.example exception). The relayer uses QTRUST_RELAYER_PRIVATE_KEY env var, falling back to QTRUST_DEPLOYER_PRIVATE_KEY (attestation.ts:23) — single key, no rotation, no multisig. If the relayer key is compromised, an attacker

could submit fraudulent attestations/migrations/CBOM registrations in any org's name (the contract records the signer, but the relayer is the signer for the write path).

File, network, and process security

File: no file reads from untrusted input in the backend. The inspector reads PEM files from a user-specified directory (file_scanner.py:12) — the user is the operator, so this is acceptable. Network: the webhook delivery (webhook.ts) has SSRF protection (blocks private ranges, requires HTTPS) but is incomplete — misses 169.254/16 (link-local), fc00::/7 (IPv6 ULA), ::ffff: mapped IPv6. The frontend makes cross-origin requests to the backend (CORS allowlist via QTRUST_CORS_ORIGINS, defaults to "*"). The SDK makes outbound HTTP requests to DID URLs (did.py) and Pinata (ipfs.py) — no SSRF risk because the URLs are constructed from the DID string (did:web:domain → https://domain/.well-known/did.json) and the DID is provided by the credential issuer, but a malicious DID could point to an internal host. Process: subprocess.run for ssh-keyscan (scanner.py:234) — list args, no shell, low risk. The CryptoScanner.scan_network uses python-nmap (scanner.py:358-371) which spawns nmap as a subprocess — nmap interprets the CIDR argument, but CIDR is a network range notation, not shell-injectable.

Dependency vulnerabilities (inferred)

No npm audit or pip-audit output is committed. No SBOM (CycloneDX, SPDX) generated. The package-lock.json is present for backend and frontend — npm audit could be run but the results are not in the repo. From the dependency list: viem 2.16 (recent, no known CVEs at audit time), web3 6.x (had a CVE in 6.0-6.3 around eth_abi decoding — recommend upgrading to 6.5+), eth-account 0.10+ (no known CVEs), pydantic 2.x (no known CVEs), @openzeppelin/contracts (submodule, version locked in foundry.lock — should be 5.x; OZ 4.x had a reentrancy in ERC777 hook but Q-Trust does not use ERC777). The inspector uses cryptography>=41.0.0 — cryptography 41-42 had a buffer overflow in X.509 OID parsing (CVE-2023-50782, fixed in 42.0); a fresh install would pull the latest, but the requirements.txt is not pinned. The planner uses torch>=2.2 — torch has had several CVEs (e.g., CVE-2024-24679 in 2.1, CVE-2024-43053 in 2.2); a fresh install would pull the latest patch. No Renovate, no Dependabot configured.

Trust boundaries and attack surfaces

Trust boundaries: (1) Browser → Frontend (Next.js) — public, untrusted. (2) Frontend → Backend (Fastify) — semi-trusted (CORS allowlist). (3) Backend → Contracts (viem) — trusted (relayer key). (4) Backend → Postgres — trusted (private network). (5) Backend → Redis — trusted (private network). (6) Backend → Planner (FastAPI) — semi-trusted (internal HTTP, no auth). (7) Backend → External webhook URLs — untrusted (SSRF risk, partially mitigated). (8) SDK → Contracts (web3.py) — trusted (caller holds their own key). (9) Inspector → External hosts (TLS/SSH scan) — untrusted (network input). Attack surfaces: the public REST API (all /v1 routes), the relay routes (signature verification only), the frontend public pages, the webhook delivery (outbound HTTP), the DID resolution (outbound HTTPS), the IPFS gateway (outbound HTTPS), the Pinata API (outbound HTTPS with credentials), the Base L2 RPC (outbound HTTPS).

Rate limiting, abuse prevention, and privilege escalation risks

Rate limiting: global 120 req/min per IP (server.ts:90-94). No per-route limits, no per-vendor limits on relay routes. A motivated attacker could submit thousands of invalid signatures per minute, each consuming CPU on recoverTypedDataAddress. The relay routes also call publicClient.readContract to fetch the on-chain nonce — one RPC call per request, so a flood would also stress the RPC provider. Abuse prevention: bodyLimit 1MB (server.ts:65). No CAPTCHA, no proof-of-work. Privilege escalation: the deployer retains REGISTRAR/MIGRATOR/AUDITOR roles post-deployment (Deploy.s.sol:78, 88-89, 30) — a

compromised deployer key could register fraudulent CBOMs or mark fraudulent migrations as verified. The timelock governance only governs DEFAULT_ADMIN_ROLE operations (pause, upgrade, role grant) — operational roles are outside the timelock.

10. Error Handling & Reliability

Exception and error handling

The backend uses try/catch in most route handlers, returning JSON {error: msg} with appropriate status codes. However, several read routes (server.ts:121-125, 136-143, 632-640) lack try/catch and will return Fastify's default 500 error on contract revert. The contract revert messages are surfaced unmodified — viem returns strings like "execution reverted: AssetNotFound(0x...)" which the backend passes through to the client. The relayer catches signature recovery failures and rethrows as a human-readable Error; the route handler then maps to 400 (for signature/nonce errors) or 422 (for contract reverts). The planner proxy (server.ts:209-225) catches fetch errors and returns 503. The Redis connection has a try/catch around the connect (server.ts:73-77) that logs a warning and sets redis=null on failure. The indexer's startIndexer has a try/catch (line 244-253) that logs and lets the API continue. But watchLive has no error handling at all (line 223-236).

Failure recovery

The indexer recovers from a Postgres outage by re-running backfill on restart (the cursor is persisted in indexer_state, so on restart it picks up from the last successful block). The API recovers from a Postgres outage mid-request by falling back to direct RPC reads (verify.ts:140-160). The API recovers from a Redis outage by disabling webhook subscriptions (server.ts:74-77). The API recovers from a planner outage by returning 503 (server.ts:222-225). However, there is no automatic recovery from a stuck relayer (waitForTransactionReceipt has no timeout — if the tx is stuck in the mempool, the request hangs until the client times out). There is no automatic recovery from a corrupted Postgres volume. There is no automatic recovery from a stale Redis subscriber set (the subscribers table never gets cleaned even if subscriber URLs go offline).

Logging and observability

Fastify logger=true (server.ts:64) logs every request as structured JSON to stdout. The indexer uses console.log/console.warn for backfill progress and failure messages. The relayer uses console.warn for webhook delivery failures. There is no structured logging framework (no pino, no winston) beyond Fastify's built-in. No correlation IDs (no X-Request-ID header propagation). No metrics endpoint (no /metrics, no Prometheus, no OpenTelemetry). No tracing. No log aggregation (no ELK, no Loki, no Datadog). The planner uses uvicorn's default logging (no config). The frontend has no error tracking (no Sentry, no LogRocket). For a project claiming production ambitions (the patent materials describe SaaS revenue), this is woefully inadequate.

Retry and timeout behavior

Webhook delivery: 3 attempts with exponential backoff ($500\text{ms} \times 2^{\text{attempt}} + 250\text{ms}$ jitter, capped at 10s) — webhook.ts:75-82. Good. Planner proxy: AbortSignal.timeout(60_000) for POST /v1/plans (server.ts:214), 30_000 for GET /v1/plans/:did (server.ts:233). Good. viem http transport timeout 30s (config.ts:28). Good. Relayer waitForTransactionReceipt: no timeout (attestation.ts:207). Bad — a stuck tx hangs the request. Indexer backfill: no retry on RPC fetch failure — a single failed getLogs call aborts the whole backfill loop (indexer.ts:208-218). The startup would then log "Indexer failed to start" (line 252) and the API would run without the indexer (degraded to direct RPC reads).

Resource cleanup

No graceful shutdown handlers. The Redis connection (server.ts:73) is never closed on process exit. The Postgres Pool (indexer.ts:25) is never ended on process exit. The publicClient.watchEvent subscription

(indexer.ts:226) is never unsubscribed on process exit. The Fastify server.listen does not register a SIGTERM handler to call server.close(). The Dockerfile uses tini (line 49) as PID 1, which handles signal forwarding and zombie reaping, but the Node process itself does not handle SIGTERM gracefully — open requests are dropped on container stop. For a stateless API this is acceptable but for the indexer it means a container restart could lose in-flight events.

Race conditions and concurrency issues

Relayer nonce race: relaySignedAttestation reads the on-chain nonce (attestation.ts:180-185), checks it against the signed nonce, then submits the tx. Two concurrent relays for the same vendor with the same nonce would both pass the check (reading the same on-chain nonce), both attempt to submit, and one would succeed while the other would revert with InvalidNonce at the contract level. The relayer would return a 422 with "Nonce mismatch" — the client would need to re-sign with the new nonce. This is correct behavior (the contract is the source of truth), but the relayer does not pre-flight with a mempool check or use a nonce manager. For high-throughput vendors this would cause retry storms. Indexer applyLog concurrency: the watchLive onLogs callback is called with a batch of logs; applyLog is called sequentially in a for-of await loop (line 229-233) — no concurrency. Good for correctness, but slow for high event rates.

Single points of failure

SPOF 1: the relayer private key. If compromised, fraudulent CBOM registrations, attestations, and migrations can be submitted. The contract records the signer (relayer), not the original requestor, for the write path — so fraudulent writes would be attributed to the relayer. Mitigation: multisig (Safe) + meta-tx framework. SPOF 2: the Postgres instance (single docker-compose service, no replica, no failover). SPOF 3: the Redis instance (single docker-compose service, no sentinel). SPOF 4: the GNN planner instance (single FastAPI process, no workers, no HA). SPOF 5: the Base L2 RPC endpoint (single URL, no fallback RPC list). SPOF 6: the Pinata IPFS pinning service (single provider, no fallback). For a production system, each of these needs redundancy.

11. Performance & Scalability

Algorithmic complexity

verify.ts:getMigrationProgress (line 319-341): $O(N)$ where N = number of migrations for the org. Calls getMigrationsByOrg (1 RPC) then getMigration per ID (N RPCs). For an org with 1000 migrations, that's 1000 sequential RPC calls. The Postgres path avoids this (indexed), but the RPC fallback does not. verify.ts:getAssetsByOrg (line 207-233, RPC fallback): same pattern, M RPCs where M = number of assets. verify.ts:getVendorAttestations (line 243-289, RPC fallback): same pattern. Contract-level: VendorRegistry.checkProductSupport (VendorRegistry.sol:356-371) loops over _attestationsByProduct[productIdHash] — $O(K)$ where $K \leq 256$ (capped). Acceptable. AssetRegistry.getAllAssetIds(offset, limit) — $O(\text{limit})$. Good. AuditRegistry.getLatestAudit — $O(1)$ (last element of array). Good.

I/O bottlenecks

Indexer backfill (indexer.ts:195-220): chunks by 2000 blocks, but each chunk's logs are processed sequentially via for-of await applyLog. No parallelism within a chunk, no parallel chunks across event types. For a contract with high event volume (e.g., 1000 events per 2000 blocks), the backfill could take minutes. The watchLive subscription uses viem's watchEvent which polls — the poll interval is implementation-defined (typically 4 seconds for HTTP transports). Backend → Planner: synchronous fetch with 60s timeout. If the planner is slow, the API request blocks. No async fire-and-forget, no result caching. Backend → Postgres: pool max=10. Under high concurrency, requests queue. Backend → Redis: lazyConnect with maxRetriesPerRequest=1. Under high concurrency, Redis operations can fail silently.

Database and network usage

Database: 5 tables, 6 indexes. Read load is dominated by /v1/orgs/:did/migrations (which queries 3 tables in parallel — migrations, audits, plus getMigrationProgress which is $N+1$ RPC calls if Postgres is down). Write load is indexer-driven (one upsert per event). No read replicas, no connection pooler (PgBouncer), no query cache. Network: every contract read on the RPC fallback path is a separate HTTP request to the RPC provider. For an org with 1000 migrations, /v1/orgs/:did/migrations with Postgres down = 1000 sequential RPC calls. No batching (viem supports multicall but it's not used here).

Memory and CPU concerns

Memory: the planner loads model.pt into process memory once at startup (server.py:43-69). The model is small (~1MB based on the architecture: $6 \rightarrow 64 \rightarrow 64 \rightarrow 32 + 2$ heads, ~50K parameters). No memory leak risk visible. The backend keeps the viem publicClient in module scope (config.ts:26) — no per-request allocation. The indexer keeps the Postgres Pool in module scope (indexer.ts:25) — max 10 connections, each ~10MB. CPU: the relayer's recoverTypedDataAddress (attestation.ts:168) is an secp256k1 ECDSA recover — ~0.5ms per call. Under 1000 relay requests/sec, that's 0.5 CPU seconds per second — one core saturated. The /v1/evaluate endpoint has a hardcoded policy loop — $O(C \times E)$ where $C=5$ clauses, E =evidence count. Trivial.

Caching

SDK DIDResolver has a 5-min TTL cache (did.py:39). No HTTP cache headers on backend responses (no Cache-Control, no ETag). No Redis-based response cache. No CDN in front of the API. The frontend uses @tanstack/react-query with staleTime: 60_000 (use-user-role.ts:32) — 1 min client-side cache. fetchIpfsJson uses cache: "no-store" (api.ts:241) — no IPFS metadata caching, every request hits the gateway. The planner does not cache plan results — every /v1/plans call recomputes the GNN forward

pass.

Synchronous vs asynchronous behavior

Backend: all route handlers are async (Promise-based). The indexer's watchLive onLogs callback is async (line 229) — viem awaits the callback. The relayer's waitForTransactionReceipt (attestation.ts:207) is async — awaits the tx receipt. No synchronous blocking calls in the backend. The planner is async (FastAPI + uvicorn). The SDK uses both sync (web3.py is sync) and async (httpx for DID resolution) — the VCVerifier has both async verify_credential and sync verify_credential_sync methods, duplicating logic. The inspector is sync (no async).

Likely scalability limits

Limits: (1) per-org asset list on-chain is an unbounded bytes32[] — gas cost grows linearly with N. An org with 10,000 assets would pay ~2M gas to read getAssetsByOrg. (2) VendorRegistry cap of 256 attestations per (product, version, algorithm) — easily hit by a motivated attacker registering 256 junk attestations. (3) The indexer's sequential per-event applyLog limits event throughput to ~100 events/sec (one Postgres upsert per event). (4) The relayer's single-key throughput limits to ~10 relay requests/sec (one secp256k1 recover + one writeContract per request). (5) The planner's single-process FastAPI limits to ~100 req/sec (one GNN forward pass per request). For production, the planner should run behind a load balancer with multiple workers. The relayer should use a Safe multisig + meta-tx framework. The indexer should batch upserts.

12. Testing & Quality

Test coverage and organization

Contracts (contracts/test/*.t.sol): 10 test files. AssetRegistry.t.sol, VendorRegistry.t.sol, MigrationRegistry.t.sol, AuditRegistry.t.sol, PolicyCommitment.t.sol, QTrustGovernance.t.sol, RevocationAnchor.t.sol, SchemaRegistry.t.sol, TrustAnchorRegistry.t.sol, Attack.t.sol. The Attack.t.sol file (277 lines) is adversarial — tests proxy upgrade authorization, pause bypass, access control, cross-registry integrity, duplicate prevention, and validation. README claims 127 tests across 10 suites; verify_all.sh:14 expects "127 tests passed". SDK (sdk/tests): test_client.py, test_did.py, test_trust.py, test_vc.py, e2e_anvil.py (full lifecycle against local anvil). Inspector (inspector/tests/test_scanner.py): 5 tests, 1 skip per README. Planner: no unit tests; benchmark.py is a benchmark script (multi-seed, honest metrics). Backend: ZERO tests (no test file, no test framework in package.json). Frontend: ZERO tests (no jest, no vitest, no playwright).

What is tested and what is not

Tested: contract happy paths, access control, cross-registry integrity (AssetNotRegistered, AssetInactive, MigratedCountExceedsOnChain, DuplicateMigration, SameAlgorithm, EmptyHash, InvalidCounts, pause bypass, proxy upgrade auth). SDK E2E: full register → attest → migrate → audit → verify cycle, gasless EIP-712 flow, integrity guards, read-only client. Inspector: basic scan_host (5 tests, 1 skip). Not tested: backend route handlers (no unit tests), backend relayer (no unit tests), indexer backfill (no integration test), indexer watchLive (no integration test), webhook delivery (no test), webhook SSRF filter (no test), /v1/evaluate (no test), /v1/credentials/verify (no test — and it returns hardcoded valid=true, so any test would fail to catch the stub), frontend components (no tests), frontend EIP-712 signing helper (no tests), planner /plan endpoint (no tests), planner cbom_to_graph (no tests). The e2e_anvil.py script is the closest thing to an integration test, but it requires a running anvil chain and is not automated in CI.

Test quality and realism

Contract tests are high quality — they use vm.prank, vm.expectRevert with selector matching, and cover adversarial cases. The Attack.t.sol file is exactly what a senior engineer wants to see. The SDK E2E is realistic (uses anvil, deploys contracts, runs the full lifecycle), but the assertions are loose — e.g., line 95-96 catches RuntimeError and asserts "reverted" in str(e), which would pass on ANY revert (including unrelated bugs). The SDK unit tests (test_client.py, test_did.py, test_trust.py, test_vc.py) were not inspected line-by-line in this audit but the file sizes suggest light coverage. The inspector test_scanner.py is 5 tests, 1 skip — light. The planner has no unit tests at all, only a benchmark. The benchmark.json results are honest (mean±std across 3 seeds) but the benchmark itself only tests the GNN on synthetic data, not real CBOMs.

Missing edge cases

Missing: (1) concurrent relay for the same vendor (nonce race). (2) webhook SSRF with 169.254.x.x, fc00::/7, ::ffff:127.0.0.1. (3) pagination with offset > total. (4) pagination with NaN offset/limit. (5) productId/version/algorithm with very long strings (gas limit). (6) metadataURI with javascript: scheme. (7) EIP-712 signature with chainId mismatch (mainnet sig submitted to testnet). (8) indexer reorg handling (no test). (9) indexer cursor gap (applyLog fails, setCursor advances). (10) planner with empty CBOM (the /plan endpoint checks for this but no test). (11) planner with all-PQC-ready assets (no migrations needed). (12) frontend with no injected wallet (mock path). (13) frontend with a wrong-chain MetaMask (e.g., Ethereum mainnet instead of Base Sepolia).

Static analysis, linting, formatting, type checking

Solidity: foundry.toml does not enable any linter. No slither config, no solhint config. TypeScript: backend tsconfig.json has strict: true, skipLibCheck: true. No eslint config for backend. Frontend has no eslint config (next lint default). Python: sdk/pyproject.toml configures ruff (select E, F, I, W, UP — not the strictest) and mypy strict=true. inspector/pyproject.toml configures ruff (same selection). Planner has NO ruff/mypy config (no [tool.ruff] or [tool.mypy] in requirements.txt — and there is no pyproject.toml for the planner at all). No pre-commit hooks (no .pre-commit-config.yaml). No code formatter config (no .prettierrc, no black config).

Code quality and maintainability

Code quality is uneven. The Solidity contracts are well-commented (Natspec on every function, custom errors, events with named fields) and idiomatic (correct use of OpenZeppelin primitives, ReentrancyGuard, Pausable, UUPS). The backend TypeScript is readable but uses `as any` casts in route handlers (e.g., server.ts:478 `preHandler: requireApiKey as any`) and ad-hoc body shape validation. The SDK Python has type hints and Pydantic models but logical bugs in vc.py. The inspector has multiple typos (opensssh, ssh-ss comment) and a wrong loader in file_scanner.py. The planner has dead code (model_v2.py) and a deprecated FastAPI startup hook. The 62KB abis.ts file is a code smell — it should be regenerated on build, not committed. Maintainability is FAIR: the code is readable, but the volume of dead code, stub endpoints, and inconsistencies (two scan_host functions, three model variants, hardcoded chainId) raises the cost of future changes.

13. DevOps / CI/CD / Deployment

Build process

Contracts: `forge build` (compiles all .sol to artifacts in contracts/out/) then `forge test` (runs all .t.sol files). Backend: `npm run build` runs `tsc -p tsconfig.json` which emits JavaScript to dist/. The Dockerfile uses `bun run build` (Dockerfile:31) which calls the same tsc — works but the README says `npm run build` (mismatch in documented tooling). Frontend: `npm run build` runs `next build` (SSR + static export). SDK: `pip install -e ./sdk` (editable install). Inspector: `pip install -e ./inspector`. Planner: no build step, runs uvicorn directly on server.py. Pilot: `python run\_pilot.py` (no build). All-builds: scripts/verify_all.sh chains all 9 build/test steps + npm audit. But verify_all.sh is not in CI.

CI/CD workflows

NONE. No .github/workflows/, no .gitlab-ci.yml, no Jenkinsfile, no CircleCI config, no Drone config. The repository has zero CI/CD configuration. scripts/verify_all.sh exists but is intended to be run manually. No push triggers anything. No PR checks. No automatic dependency updates (no Dependabot, no Renovate). This is the single most significant DevOps gap — every merge to main is unverified, and the 127 contract tests + 5 SDK tests + 5 inspector tests are only run when a developer remembers to run them locally.

Docker and containerization

docker-compose.yml defines 5 services: api (qtrust-backend:0.1.0, port 3001), webhook (same image, command bun run dist/services/webhook.js), postgres (postgres:16-alpine, port 5433 on host), planner (qtrust-planner:0.1.0, port 8000), redis (redis:7-alpine, port 6379). backend/Dockerfile is multi-stage (base → deps → build → runtime) with tini as PID 1 and a HEALTHCHECK on /health. planner/Dockerfile is single-stage python:3.11-slim with no healthcheck. No frontend Dockerfile. No frontend docker-compose service. The planner Dockerfile COPYs model.pt (line 10) — if model.pt is missing (gitignored), the build fails. The api image still has src/ copied in (Dockerfile:43) — not strictly necessary at runtime since dist/ is used, but harmless. Both images lack SBOM generation, lack image signing (cosign), and lack base-image pinning (no digest pinning).

Deployment configuration

No Terraform, no Pulumi, no Helm chart, no Kubernetes manifests. The only deployment artifact is docker-compose.yml, which is suitable for a single-host deployment but not for production HA. README mentions "Vercel deploy" as a remaining credential-dependent task (README:149) — no Vercel config in the repo. No secrets manager integration (no Vault, no AWS Secrets Manager, no Doppler). Secrets are passed via env_file: .env (docker-compose.yml:21,46). No SSL/TLS termination at the API level (no nginx, no Caddy, no Traefik) — the API listens on plain HTTP port 3001.

Infrastructure assumptions

Assumes: a host running Docker 24+ with compose v2; network access to Base Sepolia RPC (or a local anvil instance on port 8545); Pinata account for IPFS pinning (optional, falls back to hash-only); MetaMask in the browser for vendor attestations (or the mock wallet for dev). For production: would need a real Base Sepolia RPC provider (Alchemy, Infura, QuickNode), a real Pinata account, a real domain for did:web resolution, a Vercel account for the frontend, and a hosted Postgres + Redis (RDS, Upstash). None of these are pre-configured.

Release process

No release process. No semantic versioning (package.json versions are 0.1.0 everywhere). No git tags (5 commits, no tags). No changelog (no CHANGELOG.md). No release notes. No rollback procedure. The contracts are upgradeable (UUPS) so contract rollback is possible via timelock-scheduled proxy upgrade, but there is no documented procedure. The backend and frontend are stateless so rollback is just redeploying a previous image — but no previous images are retained (no container registry).

Production-readiness concerns

NOT production-ready. Concrete blockers: (1) no CI/CD — every merge is unverified. (2) no automated tests for backend or frontend. (3) stub endpoints that return hardcoded values (see § 4). (4) dev hole when API_KEYS unset. (5) webhook system is non-functional. (6) no monitoring, no log aggregation, no alerting. (7) no secrets manager. (8) no rollback procedure. (9) no load testing. (10) planner Dockerfile cannot build from fresh clone. (11) frontend has not been deployed. (12) no real deployment beyond anvil. (13) no SLA, no SLO, no runbook. (14) no incident response plan. (15) no security review (no OpenZeppelin, Trail of Bits, or equivalent audit).

14. Git & Project Hygiene

.gitignore

The .gitignore covers Python (__pycache__, *.pyc, .pytest_cache, .ruff_cache, *.egg-info, .venv, venv), Node (node_modules, .next, dist, *.tsbuildinfo), Foundry (contracts/out, contracts/cache, contracts/broadcast), env files (.env, .env.* with !env.example exception — correct pattern), planner models (planner/results/*.pt, planner/model.pt), and local tooling (.claude, .vscode, .ipynb_checkpoints). The .gitignore is reasonable. However: (1) planner/model.pt IS tracked (the file exists in the repo) — this contradicts the .gitignore rule. It was likely force-added at some point. (2) No exclusion for *.log files (the anvil logs from verify_all.sh would be committed if not redirected). (3) No exclusion for .DS_Store (macOS). (4) No exclusion for IDE files beyond .vscode (no .idea).

Tracked and generated artifacts

Tracked: backend/package-lock.json (correct), frontend/package-lock.json (correct), contracts/foundry.lock (correct), planner/model.pt (incorrect — should be in LFS or fetched on build), backend/src/lib/abis.ts (62KB generated code — should be regenerated on build, not committed), sdk/qtrust/contracts.py (also generated — same concern). Generated: contracts/out/ (gitignored), contracts/cache/ (gitignored), backend/dist/ (gitignored), frontend/.next/ (gitignored). Submodules: contracts/lib/forged-std and contracts/lib/openzeppelin-contracts (standard Foundry submodules).

Repository hygiene

5 commits on the default branch (main). No tags. No branches beyond main. No PRs. No issues. No releases. Commit messages are descriptive and follow conventional commits style ("refactor:", "fix:", "feat:", "Initial commit:"). No commit signing (no GPG, no SSH). No CONTRIBUTING.md. No CODE_OF_CONDUCT.md. No LICENSE file at the repo root (LICENSE is mentioned in package.json files as MIT but no LICENSE file exists — the only license reference is in pyproject.toml files: license = { text = "MIT" }). No SECURITY.md. No .github/ directory at all (no Issue templates, no PR template, no Dependabot config, no CodeQL config).

Documentation quality

docs/ contains 11 PHASE_*.md files (one per phase), QTrust_Implementation_Guide.md (large), demo_run_of_show.md, playbook_review.md, PROMPT.md (the original Qwen 2.5 master build prompt — reveals the AI-generated origin), PATENT/ (draft_claims.md, filing_checklist.md, invention_disclosure.md, prior_art_survey.md), and assessments/ (three long-form assessments including two prior comprehensive audits — QTrust_Comprehensive_Assessment_humoge7502.md at ~1700 lines and QTrust_Comprehensive_Assessment_Post_P0.md at ~3500 lines). The README is thorough (150 lines) with honest "Known limitations" section. Documentation quality is actually excellent for a project at this stage — better than the code quality, in fact.

Potential accidentally committed secrets or unnecessary files

No real secrets. The anvil dev key (0xac09...ff80) in 6 files is the well-known Account #0 test key — not a real secret, but a pattern anti-recommendation in .env.example and pilot/run_pilot.py. No .env file committed (only .env.example). No API keys. No private mainnet keys. The model.pt file is 6MB and is tracked — it should be in Git LFS or fetched from a release artifact, not committed as a regular file (bloats every clone). No .DS_Store. No *.log. No editor swap files. The docs/assessments/ directory contains 3 large markdown files totaling ~5,400 lines of meta-assessment — the team has been generating extensive meta-docs instead of fixing issues, which is itself a form of technical debt.

15. Detailed Findings Table

The following table lists all 24 findings with severity, file:line, function/class, why it matters, evidence, remediation, and estimated implementation difficulty. Severities: Critical (data loss / active security risk), High (security risk / broken functionality), Medium (correctness bug / missing control), Low (quality issue), Informational (style / maintainability).

F1	Critical	backend/src/server.ts :503-521 /v1/credentials/verify	Endpoint returns {valid:true, revoked:false} for ANY input. Misleads API consumers into trusting unverified VCs. Evidence: Line 517: `valid: true` hardcoded; line 519 note says "Full VC verification with DID resolution is available via the Python SDK" – but the SDK also does not verify (see F8). Fix: Either implement real VC verification (call SDK VCVerifier with the presentation) or remove the endpoint and route consumers to the SDK directly. Effort: 4h
F2	Critical	backend/src/server.ts :99-102 requireApiKey	When QTRUST_API_KEYS is unset, all /v1/write/* routes are open. Production deployments with an empty API_KEYS list would accept unauthenticated writes. Evidence: Line 99: `if (!API_KEYS.length) { return done(); }` – comment says "dev mode, allow writes". Fix: Fail-closed: `if (!API_KEYS.length && process.env.NODE_ENV !== "production") return done(); else if (!API_KEYS.length) return reply.status(401).send({error: "API keys not configured"});` Effort: 15min
F3	High	backend/src/services/ webhook.ts:88 fanOut	fanOut is exported but NEVER imported by any other file (grep-confirmed). The webhook subscribe/unsubscribe endpoints maintain Redis sets nobody reads. The webhook system is non-functional theater. Evidence: <code>grep -r "fanOut deliverWebhook"</code> shows only the definition in <code>webhook.ts</code> – no callers. The delivery path uses <code>QTRUST_WEBHOOKS</code> env var (<code>webhook.ts:89</code>). Fix: Wire fanOut into the indexer's watchLive onLogs callback (<code>indexer.ts:229-235</code>) so that contract events actually trigger webhook delivery. Or remove the Redis-backed subscribe system and document the env-var approach. Effort: 4h

F4	Low	backend/package.json: 17 dependencies.bullmq	BullMQ is listed as a dependency but is never imported in any backend source file. README claims "BullMQ webhooks" — inaccurate. Increases install footprint and attack surface for no benefit. Evidence: <code>grep -r "bullmq\ BullMQ\ Queue\ Worker" in backend/src/</code> — no matches in source files, only in package.json and package-lock.json. Fix: Remove bullmq from backend/package.json dependencies. Re-run npm install to update package-lock.json. Effort: 5min
F5	High	planner/Dockerfile:10 COPY model.pt	The Dockerfile COPYs model.pt, but .gitignore:28 excludes planner/model.pt. A fresh clone has no model.pt and `docker build` fails. Evidence: planner/Dockerfile line 10: `COPY model.pt model.pt`. .gitignore line 28: `planner/model.pt`. The model.pt file IS tracked in the repo currently (force-added), but the .gitignore rule means subsequent re-trains would not commit a new model. Fix: Either remove the .gitignore rule for model.pt (track it explicitly, accept the 6MB repo bloat) or fetch the model from a release artifact in the Dockerfile (e.g., <code>curl -L https://github.com/.../releases/download/v0.1/model.pt -o model.pt</code>). Effort: 30min
F6	High	planner/qtrust_planner/model_v2.py MigrationGNN v2	175-line "patent-grade hybrid" model referenced in patent materials but NEVER imported by train.py, predict.py, or server.py — all three import only model.py (v1). v2 is dead code. Evidence: <code>grep "from .model_v2\ from qtrust_planner.model_v2" — no matches. train.py:37 imports `from .model import MigrationGNN`. predict.py:31 same. server.py:47 same.</code> Fix: Either wire model_v2.py into the training/serving pipeline (make it the default variant) or remove it. If retained for patent purposes, add a test that loads a v2 checkpoint to catch silent <code>strict=False</code> weight loading (predict.py:162). Effort: 4h to wire, 1h to remove

F7	High	contracts/src/QTrustGovernance.sol:74-86 schedulePause / scheduleUnpause	Both functions always encode AssetRegistry.pause signature regardless of registryIndex. Works today only because all 4 registries inherit Pausable and have identical pause() signatures. Will silently produce the wrong call for any future contract with a different pause signature. Evidence: QTrustGovernance.sol line 77: <code>`bytes memory data = abi.encodeCall(AssetRegistry.pause, ());`</code> – registryIndex is used only at line 76 to pick the target address, not the function signature. Fix: Define an IPausable interface with pause() and unpause(), and encode against that. Or switch on registryIndex to encode each registry's own pause(): if registryIndex==0: encodeCall(AssetRegistry.pause); else if 1: encodeCall(VendorRegistry.pause); etc. Effort: 1h
F8	High	sdk/qtrust/vc.py:319-379 VCVerifier.verify_credential	Does not verify the Ed25519 signature. Line 354 comment: "Signature verification delegated to holder" — the actual <code>_ed25519_verify</code> function is NEVER called. The verifier returns <code>valid=true</code> for any VC with a proof. Evidence: vc.py line 350-354: <code>`auth_key = self.resolver.get_authentication_key(issuer_doc); public_key = auth_key.get(...) or ...; # For now, just check signature format; explanation_parts.append("Signature verification delegated to holder")`</code>. Fix: Call <code>_ed25519_verify(message, signature, public_key)</code> after extracting the public key from the DID document. Handle the case where the DID key is in <code>publicKeyJwk</code> format (convert to raw bytes for pynacl). Effort: 4h
F9	High	sdk/qtrust/vc.py:333-340 VCVerifier.verify_credential (except handler)	On DID resolution failure, the except handler returns <code>`valid=not expired and not revoked`</code> — but <code>`revoked`</code> is referenced before assignment in this scope. Python would raise <code>NameError</code>, defeating the try/except. Evidence: vc.py line 333: <code>`return VCVerificationResult(valid=not expired and not revoked, ...)`</code> — <code>`revoked`</code> is not defined in the except handler scope (it is only defined later in the try-success path at line 367). Fix: Initialize <code>`revoked = False`</code> at the top of the method, before the try/except. Effort: 5min

F1 0	Medium	backend/src/services/ attestation.ts:127, 233, 334 EIP712_DOMAIN constants	chainId hardcoded to 84532 (Base Sepolia) in all three EIP712_DOMAIN constants. If QTRUST_USE_MAINNET=true (CHAIN.id=8453), the EIP-712 domain still claims 84532, so signatures signed by mainnet wallets would be unverifiable. Evidence: attestation.ts line 127: `chainId: 84532`. line 233: `chainId: 84532`. line 334: `chainId: 84532`. config.ts line 15-23 supports QTRUST_USE_MAINNET. Fix: Replace hardcoded 84532 with CHAIN.id (imported from config.ts). Re-test the EIP-712 flow on both testnet and mainnet. Effort: 30min
F1 1	Medium	backend/src/services/ attestation.ts:36-38 walletClient chain	The relayer walletClient is hardcoded to baseSepolia chain. If QTRUST_USE_MAINNET=true, the relayer will sign and submit to baseSepolia — silent breakage on mainnet deployment. Evidence: attestation.ts line 36: `chain: baseSepolia`. config.ts line 15-23 supports USE_MAINNET. Fix: Import CHAIN from config.ts and use it: `chain: CHAIN`. Same for publicClient at line 39-42. Effort: 15min
F1 2	Medium	backend/src/services/ indexer.ts:131-177 applyLog + setCursor	applyLog and setCursor are NOT wrapped in a transaction. If Postgres crashes between the upsert and the cursor advance, the next backfill re-applies the same log. Mostly idempotent for INSERT/UPDATE, but the revoked toggle on AttestationRevoked could double-toggle. Evidence: indexer.ts line 216: `await applyLog(spec, log as Log);` then line 232: `await setCursor(spec.key, (log.blockNumber ?? 0n) + 1n);` – no BEGIN/COMMIT. Fix: Wrap applyLog + setCursor in a single Postgres transaction: `const client = await pool.connect(); await client.query("BEGIN"); try { await applyLog(client, ...); await setCursor(client, ...); await client.query("COMMIT"); } catch { await client.query("ROLLBACK"); } finally { client.release(); }` Effort: 2h

F1 3	Medium	backend/src/services/ webhook.ts:22-46 isPublicHttpsUrl	SSRF filter blocks localhost, 0.0.0.0, .local, .internal, 127/8, 10/8, 192.168/16, 172.16-31, ::1, fe80:. MISSES: 169.254.0.0/16 (link-local), fc00::/7 (IPv6 ULA), ::ffff:127.0.0.1 (IPv4-mapped IPv6), fec0: (IPv6 site-local). Evidence: webhook.ts line 33-39: regex list. No check for 169.254, no check for fc00::/7, no check for ::ffff: prefix. Fix: Add the missing ranges: <code>`/^169\..254\./.test(host)`</code>, <code>`host.startsWith("fc")    host.startsWith("fd")`</code> for ULA, <code>`/^::ffff:./.test(host)`</code> for IPv4-mapped. Effort: 30min
F1 4	Medium	inspector/qtrust_insp ector/file_scanner.py :25 scan_pem_files	Uses x509.load_der_x509_certificate on PEM files. A .pem file contains PEM-armored base64, not raw DER. The cryptography library provides load_pem_x509_certificate for this. The current code silently produces no findings for PEM files. Evidence: file_scanner.py line 25: <code>`cert = x509.load_der_x509_certificate(content)`</code> – content is a .pem file bytes. Fix: Use load_pem_x509_certificate for .pem/.crt/.cert files. Use load_der_x509_certificate only for .der files. Detect format by attempting PEM first (try load_pem_x509_certificate), falling back to DER. Effort: 1h
F1 5	Medium	pilot/run_pilot.py:78 step1_scan	On the no-findings path, line 78 returns <code>{"cbom": cbom, ...}</code> but the variable <code>`cbom`</code> is only constructed in the else branch at line 80-94. The no-findings path builds <code>`result`</code> (a CBOM) at line 57-77, not <code>`cbom`</code>. NameError on the no-findings path. Evidence: pilot/run_pilot.py line 57-77: builds <code>`result = CBOM(...)`</code>. Line 78: <code>`return {"cbom": cbom, "synthetic": True}`</code> – <code>`cbom`</code> is undefined. Fix: Rename <code>`result`</code> to <code>`cbom`</code> at line 57, or change line 78 to <code>`return {"cbom": result, "synthetic": True}`</code>. Effort: 5min

F1 6	Medium	backend/src/server.ts :526-572 /v1/revocation, /v1/policies, /v1/schemas, /v1/trust-anchors	All four endpoints return hardcoded 0x000000...000 hashes and "note" strings. They do not query the deployed RevocationAnchor, PolicyCommitment, SchemaRegistry, or TrustAnchorRegistry contracts. Misleading: the API surface implies functionality that is not implemented. Evidence: server.ts line 530: `current_root: "0x0000000000000000000000000000000000000000000000000000000000000000".` Line 543: `policy_hash: "0x000..."`. Line 558: returns only `{schema_id, note}`. Line 570: `accredited: false` hardcoded. Fix: Wire each endpoint to a viem readContract call against the deployed contract. Add the contract addresses to config.ts:CONTRACTS. Regenerate ABIs to include the trust-infra contracts. Effort: 1 day
F1 7	Medium	.github/workflows/ (none)	No CI/CD pipeline. Every merge to main is unverified. The 127 contract tests + 5 SDK tests + 5 inspector tests are only run when a developer remembers locally. Evidence: No .github/workflows/ directory exists. No .gitlab-ci.yml. No Jenkinsfile. Fix: Add .github/workflows/ci.yml that runs scripts/verify_all.sh on every push and PR. Cache Foundry, npm, and pip installs. Run on ubuntu-latest with Node 20, Python 3.11, Foundry master. Effort: 4h
F1 8	Low	backend/.env.example: 12, pilot/run_pilot.py:43 anvil dev private key	The anvil dev key 0xac0974...ff80 appears in committed files including .env.example (a template users copy to .env) and pilot/run_pilot.py. Not a real secret, but a pattern anti-recommendation: a user copying .env.example verbatim would use the anvil key in production. Evidence: backend/.env.example line 12: `QTRUST_RELAYER_PRIVATE_KEY=0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbde5efcae784d7bf4f2ff80`. pilot/run_pilot.py line 43: same value via os.environ.setdefault(). Fix: Replace the value in .env.example with a placeholder `0xYOUR_PRIVATE_KEY_HERE`. Keep the anvil key in test files (e2e_anvil.py, run_e2e.sh, verify_all.sh) — those are acceptable. Effort: 15min

F1 9	Low	backend/, frontend/ (no test files)	No automated tests for the backend or frontend. No test framework in backend/package.json devDependencies (only @types/*, tsx, typescript). No jest/vitest config in frontend. Manual testing only. Evidence: backend/package.json devDependencies: @types/node, @types/pg, tsx, typescript – no test runner. frontend/package.json: no test script, no jest/vitest. Fix: Add vitest to both backend and frontend. Write unit tests for requireApiKey, parseAssetId, the EIP-712 nonce check, and each route's input validation. Write component tests for the attestation form and the verification page. Effort: 2 days
F2 0	Low	backend/src/services/ verify.ts:319-341 getMigrationProgress	O(N) RPC calls — one getMigration per migration ID. For an org with 1000 migrations, that's 1000 sequential RPC reads when Postgres is down. Evidence: verify.ts line 322: `const migrationIds = (await migrationRegistry.read.getMigrationsByOrg([orgAddress])) as readonly 0x\${string}[];`. Line 326-329: `for (const migId of migrationIds) { const m = await migrationRegistry.read.getMigration([migId]); ... }`. Fix: Use viem multicall (readContracts with multicall: true) to batch all getMigration calls into one RPC. Or document that the Postgres path is required for orgs with >100 migrations. Effort: 4h
F2 1	Informational	inspector/qtrust_insp ector/scanner.py:466, 279, 149 CryptoScanner	Multiple typos: "opensssh" (3 s's, line 466), "ssh-ss" comment (line 279, should be "ssh-rsa"), trailing comment block at line 149 (indentation misleading). Evidence: scanner.py line 466: `vendor: "opensssh"`. Line 279: `# length of "ssh-ss" string`. Fix: Fix typos. Remove the trailing comment block at line 149. Effort: 15min
F2 2	Informational	README.md:28, 103, verify_all.sh:14 test count mismatch	README quick-start says "49/49 pass (5 suites)" but the status table says "127/127" and verify_all.sh:14 expects "127 tests passed". Inconsistent documentation. Evidence: README.md line 28: `forge test # 49/49 pass (5 suites)`. README.md line 103: `forge test 49/49, deployment with governance OK`. README.md line 108: `127/127`. verify_all.sh line 14: `grep -q "127 tests passed"`. Fix: Update README quick-start to say "127/127 pass (10 suites)" or remove the test count from the quick-start and reference the status table only. Effort: 5min

F23	Informational	planner/server.py:143 @app.on_event("startup")	FastAPI deprecated <code>@app.on_event("startup")</code> in favor of lifespan context managers. Will eventually stop working in a future FastAPI release. Evidence: planner/server.py line 143: <code>@app.on_event("startup")`</code>.</p> <p><b>Fix:</b> Migrate to lifespan: <code>`from contextlib import asynccontextmanager; @asynccontextmanager; async def lifespan(app): _load_model(); yield; app = FastAPI(lifespan=lifespan)`</code> Effort: 15min
F24	Informational	frontend/package.json :19 reactflow ^11.11.4	reactflow v11 is deprecated; the package was renamed to <code>@xyflow/react</code> for v12. Will eventually stop receiving updates. Evidence: frontend/package.json line 19: <code>`"reactflow": "^11.11.4"`</code>. Fix: Migrate to <code>@xyflow/react</code> v12 (API is largely compatible). Update imports from <code>`reactflow`</code> to <code>`@xyflow/react`</code>. Effort: 2h

Table 1. Prioritized findings (24 total): 2 Critical, 7 High, 8 Medium, 4 Low, 3 Informational. Each row cites exact file:line and function/class, with evidence and remediation.

16. Positive Findings

The codebase has several engineering practices worth preserving. These should be maintained as the project evolves.

Strong engineering practices

- OpenZeppelin AccessControl + TransparentUpgradeableProxy + UUPS upgradeability + TimelockController governance — the contract layer uses industry-standard, audited primitives. The deployer renounces DEFAULT_ADMIN_ROLE and hands it to a 2-day timelock (Deploy.s.sol:163-181), so no single key can pause, upgrade, or grant roles without a public notice period. This is the correct production posture.
- EIP-712 gasless attestation flow — vendors sign typed data off-chain with their wallet, the relayer verifies the signature and submits on-chain, the contract records the SIGNER as the vendor (not the relayer). Vendors never need to hold gas or run a node. The contract re-verifies the signature and the nonce before recording (AssetRegistry.sol:128-143, VendorRegistry.sol:143-166, MigrationRegistry.sol:141-163).
- Anti-replay via on-chain nonces — checked both in the relayer (attestation.ts:180-190) and in the contract (AssetRegistry.sol:136, VendorRegistry.sol:156, MigrationRegistry.sol:156). The relayer reads the on-chain nonce and compares against the signed nonce before submitting; the contract re-checks before recording. Defense in depth.
- Anti-SSRF in webhook delivery — webhook.ts:22-46 blocks localhost, 0.0.0.0, .local, .internal, 127/8, 10/8, 192.168/16, 172.16-31, ::1, fe80:.. Requires HTTPS. (See F13 for the missing ranges, but the foundation is correct.)
- Cross-registry integrity — MigrationRegistry._recordMigration calls AssetRegistry.verifyAsset before recording (MigrationRegistry.sol:222-224). AuditRegistry.postAudit rejects assetsMigrated > on-chain migration count (AuditRegistry.sol:87-90). This prevents an auditor from claiming more migrations than actually happened on-chain.
- Contract-level DoS protection — VendorRegistry.MAX_ATTESTATIONS_PER_PRODUCT = 256 (VendorRegistry.sol:28) caps the number of attestations per (productId, version, algorithm) tuple, preventing a single product from accumulating unbounded attestations.
- Idempotent indexer upserts — INSERT ... ON CONFLICT DO UPDATE (indexer.ts:42-47, 55-60, 67-70, 78-83, 91-93, 101-106, 114-119). Replays from cursor gaps produce the same row, no double-counting.
- Graceful degradation — when Postgres is unavailable, the API falls back to direct RPC reads (verify.ts:140-160). When Redis is unavailable, webhook subscriptions are disabled but the API still serves (server.ts:74-77). When the planner is unavailable, /v1/plans returns 503 with a helpful message (server.ts:222-225).
- Honest benchmark reporting — README:130-140 explicitly retracts earlier inflated GNN claims ("Kendall τ 0.924 was not reproducible from the code") and replaces them with multi-seed mean $\pm$ std. The benchmark.json reports 4 baselines (random, heuristic, GNN-MSE, GNN-ListMLE) with honest metrics. This is rare and commendable.
- Adversarial contract tests — Attack.t.sol (277 lines) tests proxy upgrade authorization, pause bypass, access control (non-admin cannot pause, non-registrar cannot register, non-migrator cannot record, non-auditor cannot post), cross-registry integrity (cannot record migration for unregistered/inactive asset, cannot post audit exceeding on-chain migrations), duplicate prevention, and validation (empty hash, same algorithm, invalid counts).

- Pausable + ReentrancyGuard on all write contracts — AssetRegistry, VendorRegistry, MigrationRegistry, AuditRegistry, RevocationAnchor all inherit both. Pause is admin-gated; reentrancy guard is on all external state-changing functions.
- Fastify bodyLimit 1MB (server.ts:65) and global rate limit 120 req/min (server.ts:90-94) — basic abuse prevention.
- Subprocess uses list args (not shell=True) for ssh-keyscan (scanner.py:234-239) — no shell injection.
- Careful nonce + signature verification on gasless relay — recoverTypedDataAddress throws on bad signatures (attestation.ts:167-177), the route handler catches and returns 400 with a clear error message (server.ts:340-343).
- Deterministic trust evaluation — TrustAssessment.compute_hash (sdk/qtrust/trust.py:58-73) produces a canonical SHA-256, so the same evidence + policy + version always produces the same hash. Reproducible.
- Cross-domain schema equivalence in SchemaRegistry — the addEquivalence function (SchemaRegistry.sol:132) allows mapping schemas across domains (e.g., "https://qtrust.dev/schemas/ppc-readiness/v1" equivalent to "https://example.com/schemas/ppc/v2"). Useful for cross-org trust.

17. Technical Debt

Existing debt

Two prior assessments in docs/assessments/ total ~5,400 lines of meta-analysis. The team has been generating extensive meta-docs instead of fixing issues. This is itself a form of debt — the cost of writing and reading these docs exceeds the cost of fixing the issues they describe.

- Three MigrationGNN variants: `model.py` (v1 GCN — used), `model_v2.py` (hybrid GCN+GATv2 — dead), `model_legacy.py` (duplicate of v1 — dead). The v1 is the only one wired into `train/predict/server`.
- Two `scan_host` functions in `inspector`: `CryptoScanner.scan_host` (`scanner.py:315` — returns dict) and module-level `scan_host` (`scanner.py:519` — returns `ScanResult`). Inconsistent API. The pilot imports the module-level one.
- Trust infra contracts deployed but backend has stub endpoints for them (`server.ts:526-572`). Either wire them up or remove them.
- BullMQ as an unused dependency (`backend/package.json:17`).
- Frontend's `NEXT_PUBLIC_DYNAMIC_ENV_ID` env var (`frontend/.env.example:11`) — Dynamic Labs SDK was removed per the post-P0 assessment, but the env var remains.
- `backend/src/lib/abis.ts` is 62KB of generated code checked into git (`sdk/qtrust/contracts.py` is similar). Should be regenerated on build, not committed.
- Two env var names for the same thing: `QTRUST_RELAYER_PRIVATE_KEY` vs `QTRUST_DEPLOYER_PRIVATE_KEY` (`attestation.ts:23` falls back). Frontend uses `QTRUST_REGISTRY_ADDRESS` (singular — likely a typo) for `assetRegistry` (`frontend/src/lib/config.ts:31`) while backend uses `QTRUST_ASSET_REGISTRY_ADDRESS`.
- Frontend `config.ts` uses `QTRUST_REGISTRY_ADDRESS` (singular, likely typo) for `assetRegistry` at line 31 — inconsistent with backend's `QTRUST_ASSET_REGISTRY_ADDRESS`.

Architectural debt

No abstraction over viem contracts in the backend — 4 contract instances duplicated in `verify.ts` (`assetRegistry`, `vendorRegistry`, `migrationRegistry`, `auditRegistry`). Adding a 5th contract requires touching `verify.ts` in 4 places (instance creation, read function, `indexer EVENTS` list, `lib/abis.ts`). A registry-of-contracts pattern would be cleaner. The trust infra contracts (`RevocationAnchor`, `PolicyCommitment`, `SchemaRegistry`, `TrustAnchorRegistry`) are deployed but not integrated with the indexer or backend — they exist as on-chain artifacts with stub API endpoints. The planner is a separate microservice but the backend has no caching of plan results — every `/v1/plans` call hits the planner. The frontend has no server-side rendering for the public verification page (`/v/[id]`) — every visitor hits the backend, even though the verification result is cacheable (the on-chain state only changes on new events).

Missing abstractions

- No shared types between backend and frontend. The frontend re-declares `AssetInfo`, `AssetVerification`, `VendorAttestationInfo`, etc. in `api.ts`. A shared types package would prevent drift.
- No shared schema validation between SDK, backend, and contracts. The ABIs are regenerated but not used for runtime validation. The SDK's Pydantic models (`schema.py`) are not shared with the backend.
- No `PolicyEngine` abstraction — `/v1/evaluate` (`server.ts:406-473`) has hardcoded policy clauses in the route handler. The SDK has a proper `TrustEvaluator` (`sdk/qtrust/trust.py`) that accepts `policy_clauses` as

a parameter — the backend should call it, not reimplement.

- No meta-transaction framework — the relayer is a single-key signer. ERC-4337 paymaster + Safe multisig would be the production pattern. The PROMPT.md mentions ERC-4337 but it is not implemented.

Fragile implementations

- inspector/scanner.py:279 comment "ssh-ss" but parses "ssh-rsa" — comment is wrong, code is correct.
- inspector/file_scanner.py:25 uses load_der on PEM files — wrong loader, silently produces no findings.
- pilot/run_pilot.py:78 references cbom before assignment on no-findings path — NameError.
- sdk/qtrust/vc.py:333 references revoked before assignment in DID except handler — NameError.
- QTrustGovernance.schedulePause/Unpause hardcode AssetRegistry.pause signature regardless of registryIndex — works only by coincidence.

Areas likely to become expensive to maintain

- The 62KB abis.ts makes backend code review painful — diffs are dominated by generated code. Switch to .gitignore + regenerate-on-build.
- The trust infra contracts are deployed but unused — either wire them up (1 day) or remove them (1 hour). The current state is the worst of both: deployed (gas spent) and unused (no value).
- The v2 GNN is referenced in patent docs but never used in production — drift risk. If a developer attempts to load a v2 checkpoint, predict.py:159-162 silently uses strict=False, loading random weights for new layers.
- The stub endpoints (/v1/credentials/verify, /v1/revocation, etc.) are a reputational risk — API consumers will trust them. The longer they exist, the harder they are to remove.
- The two scan_host functions in the inspector will diverge over time — one will get a bug fix the other does not.

18. Production Readiness

Assessment

Verdict: Development-ready. NOT Staging-ready. NOT Production-ready.

The contracts are staging-ready (well-designed, 127 tests, adversarial cases, audited primitives). The backend, frontend, SDK, inspector, and planner are development-ready (functional against local anvil, but no CI, no tests for backend/frontend, stub endpoints, dev holes). The pilot is demo-ready (works against local anvil with the synthetic CBOM fallback). The system as a whole is dev-ready: a developer can clone the repo, run `docker compose up`, start anvil, deploy the contracts, and run the pilot end-to-end. But no aspect of the system is ready for staging or production.

Concrete evidence

- No CI/CD pipeline — every merge is unverified. No `.github/workflows/` exists.
- No automated tests for backend (zero test files, no test framework in `package.json`) or frontend (zero test files, no `jest/vitest`).
- Stub endpoints return hardcoded values: `/v1/credentials/verify` returns `{valid:true, revoked:false}` for any input (server.ts:503-521). `/v1/revocation` returns `0x00...00` (server.ts:526-533). `/v1/policies` returns `0x00...00` (server.ts:538-549). `/v1/schemas` returns only `{schema_id, note}` (server.ts:554-560). `/v1/trust-anchors` returns `{accredited: false}` (server.ts:565-572). These would actively mislead production users.
- Dev hole: when `QTRUST_API_KEYS` is unset, all `/v1/write/*` routes are open (server.ts:99-102).
- Webhook system is non-functional — `fanOut` is never called (webhook.ts:88). `Subscribe/unsubscribe` maintain Redis sets nobody reads.
- BullMQ dependency unused (backend/package.json:17). README claims "BullMQ webhooks" — inaccurate.
- v2 GNN (`model_v2.py`) is dead code despite patent claims.
- No monitoring, no log aggregation, no alerting. `Fastify logger=true` is the only observability.
- No secrets manager. All secrets in `.env` files.
- No rollback strategy. UUPS allows contract rollback via `timelock` upgrade, but no documented procedure.
- No load testing. The sequential per-event indexer `applyLog` limits throughput to ~100 events/sec.
- Planner Dockerfile cannot build from fresh clone (`model.pt` is `gitignored`).
- Frontend has not been deployed (no Vercel config, no Dockerfile).
- No real deployment — anvil only. README:144 acknowledges "live Base Sepolia deployment is pending external credentials".
- No SLA, no SLO, no runbook, no incident response plan.
- No security review — no OpenZeppelin, Trail of Bits, or equivalent audit. The patent materials in `docs/PATENT/` suggest commercial intent, which makes the lack of audit more concerning.

Subsystem-by-subsystem readiness

- **Smart contracts:** Staging-ready. 127 tests, adversarial cases, OZ primitives, timelock governance. Need: formal audit (Trail of Bits), Base Sepolia deployment, Basescan verification.
- **Backend API (read routes):** Dev-ready. Works against anvil. Need: tests, monitoring, OpenAPI spec, per-route rate limits.
- **Backend API (write routes):** Dev-ready with caveat. The dev hole (F2) must be closed before any non-local deployment.
- **Backend relayer:** Dev-ready. EIP-712 flow is sound. Need: multisig (Safe), waitForTransactionReceipt timeout, retry on mempool stuck.
- **Backend indexer:** Dev-ready. Idempotent upserts. Need: transactional applyLog+setCursor (F12), reorg handling, alerting on cursor stall.
- **Backend webhooks:** NOT ready. fanOut is dead code (F3). Delivery is env-var-only.
- **Frontend:** Dev-ready. Next.js 16 + React 19 + viem. Need: tests, Vercel deploy, SSR for /v/[id], error tracking (Sentry).
- **Python SDK:** Dev-ready. Full lifecycle tested. Need: VCVerifier actually verifies (F8, F9), mainnet chainId support.
- **Inspector:** Dev-ready with bugs. file_scanner.py uses wrong loader (F14). Need: typos fixed, format-aware loader.
- **Planner:** Dev-ready. v1 GNN works. Need: remove v2 dead code (F6), fix Dockerfile (F5), lifespan context (F23), tests.
- **Pilot:** Demo-ready. Works against anvil with synthetic CBOM fallback. Need: fix cbom-before-assignment (F15).

19. Recommended Roadmap

Prioritized remediation plan with dependencies. P0 = immediate (same week), P1 = short-term (1-2 weeks), P2 = medium-term (1-2 months), P3 = long-term (3-6 months).

P0 — Immediate fixes (same week, <1 day each)

- F2: Close the API key dev hole in requireApiKey (server.ts:99-102). Fail-closed in production. 15 min.
- F1: Either implement /v1/credentials/verify properly (call the SDK VCVerifier) or remove the endpoint and route consumers to the SDK directly. 4h.
- F8 + F9: Fix VCVerifier.verify_credential — initialize revoked=False at the top, and actually call _ed25519_verify after extracting the public key from the DID document. 4h.
- F15: Fix run_pilot.py step1_scan cbom-before-assignment (rename `result` to `cbom` or change line 78). 5 min.
- F5: Fix planner/Dockerfile to either fetch model.pt from a release artifact or fail fast with a clear error. 30 min.
- F3: Wire webhook fanOut into the indexer's watchLive onLogs callback (indexer.ts:229-235) so contract events actually trigger webhook delivery. Or remove the Redis-backed subscribe system and document the env-var approach. 4h.

P1 — Short-term improvements (1-2 weeks)

- F17: Add GitHub Actions workflow running scripts/verify_all.sh on every push and PR. Cache Foundry, npm, and pip installs. 4h.
- F19: Add backend unit tests (vitest) for requireApiKey, parseAssetId, the EIP-712 nonce check, and each route's input validation. Add frontend component tests (vitest + @testing-library/react). 2 days.
- F10 + F11: Make EIP-712 chainId dynamic (read from CHAIN.id in config.ts) in attestation.ts:127, 233, 334. Make the relayer's walletClient chain follow USE_MAINNET (attestation.ts:36). 45 min.
- F7: Fix QTrustGovernance.schedulePause/Unpause to encode the correct registry's pause signature (define IPausable interface or switch on registryIndex). 1h.
- F4: Remove BullMQ from backend/package.json. 5 min.
- F6: Either wire model_v2.py into the training/serving pipeline (make it the default variant) or remove it. If retained for patent purposes, add a test that loads a v2 checkpoint to catch silent strict=False weight loading. 4h to wire, 1h to remove.
- F16: Wire /v1/revocation, /v1/policies, /v1/schemas, /v1/trust-anchors to viem readContract calls against the deployed contracts. Add the contract addresses to config.ts:CONTRACTS. Regenerate ABIs. 1 day.
- F14: Fix file_scanner.py to use load_pem_x509_certificate for .pem/.crt/.cert files (load_der only for .der). 1h.
- F18: Replace the anvil dev key value in backend/.env.example with a placeholder `0xYOUR\_PRIVATE\_KEY\_HERE`. 15 min.
- Replace /v1/evaluate inline policy with a call to the SDK TrustEvaluator (or import the policy_clauses from a config file). 4h.
- Migrate frontend from reactflow ^11 to @xyflow/react (F24). 2h.

P2 — Medium-term architectural work (1-2 months)

- Add JSON schema validation (@fastify/type-provider-zod or ajv) to all POST routes. Replace ad-hoc body shape checks. 1 day.
- F12: Wrap indexer applyLog + setCursor in a single Postgres transaction. 2h.
- Switch indexer schema to a proper migration tool (Prisma or alembic) for schema evolution. 1 day.
- Add per-route and per-IP rate limiting with Redis buckets (replace global 120/min). 1 day.
- Add OpenTelemetry tracing + structured logging (pino) + a /metrics endpoint for Prometheus. 2 days.
- Add a frontend Dockerfile and integrate frontend into docker-compose. 4h.
- Add a backend integration test that drives the full lifecycle against anvil (register CBOM, attest, migrate, audit, verify) — would have caught F15. 1 day.
- Add contract-level fuzzing (echidna or foundry invariant tests). 2 days.
- F13: Complete the SSRF filter — add 169.254/16, fc00::/7, ::ffff: mapped IPv6. 30 min.
- F20: Use viem multicall for the RPC fallback paths (getMigrationProgress, getAssetsByOrg, getVendorAttestations). 4h.
- Add waitForTransactionReceipt timeout in the relayer (attestation.ts:207). 30 min.
- Add graceful shutdown handlers (SIGTERM → server.close + pool.end + redis.quit + watcher.unsubscribe). 4h.
- Generate abis.ts and contracts.py on build (gitignore them, add to verify_all.sh). 2h.

P3 — Long-term enhancements (3-6 months)

- Deploy to Base Sepolia and verify on Basescan. Requires faucet ETH. 1 day.
- Replace the relayer single-key with a Safe multisig + relayer meta-transaction framework. 1 week.
- Implement an ERC-4337 paymaster so non-crypto customers can pay in fiat via Stripe (per PROMPT.md design notes). 2 weeks.
- Conduct a formal contract audit (OpenZeppelin, Trail of Bits, or equivalent). 2-4 weeks.
- Add real-world CBOM evaluation for the GNN — currently trained and evaluated on synthetic data only. The README acknowledges this. 2-4 weeks.
- Replace Pinata with a self-hosted IPFS node or a content-addressed storage layer with stronger availability guarantees. 1 week.
- Implement proper VC revocation using the RevocationAnchor contract (currently the verifier is a stub). 1 week.
- Add server-side rendering for the public verification page (/v/[id]) with cache-control headers. 2 days.
- Set up a staging environment (Base Sepolia + staging Postgres + staging Redis + staging frontend on Vercel). 1 week.

Dependencies between recommendations

P0 fixes (F1, F2, F3, F5, F8, F9, F15) can all proceed in parallel — they touch different files and have no inter-dependencies. P1-17 (CI) is a prerequisite for P2 (schema validation, integration tests) and P3 (live deploy). P1-10/P1-11 (chainId/mainnet fixes) are prerequisites for P3-1 (Base Sepolia deploy) — the EIP-712

flow would break on mainnet without them. P2-fuzzing is a prerequisite for P3-formal-audit — auditors expect fuzzing results. P3-paymaster depends on P3-multisig (the paymaster needs a funded multisig). P3-staging depends on P2-monitoring (you cannot deploy to staging without monitoring).

20. Final Senior-Engineer Assessment

Overall assessment

Architecture quality: GOOD for contracts, FAIR for backend, WEAK for SDK. The contract layer (proxy + UUPS + AccessControl + timelock governance + EIP-712 + cross-registry integrity) is industry-standard and would pass a senior-engineer review. The backend is reasonable (Fastify + viem + Postgres indexer with graceful degradation), but stub endpoints and dead code (fanOut, BullMQ) erode confidence. The SDK has a logical bug in VCVerifier (does not verify signatures) and a NameError in the DID except handler — these would be caught by a single end-to-end test, which does not exist.

Code quality: GOOD for contracts, FAIR for backend, FAIR for SDK, WEAK for inspector. The Solidity is well-commented (Natspec on every function, custom errors, events with named fields) and idiomatic. The backend TypeScript uses strict mode but `as any` casts in route handlers and ad-hoc body validation. The SDK has type hints and Pydantic models but logical bugs in vc.py. The inspector has multiple typos (opensssh, ssh-ss comment) and a wrong loader in file_scanner.py.

Security posture: MIXED. Contracts are well-designed (timelock, roles, EIP-712 anti-replay, anti-SSRF). But the backend has a dev hole (F2) and a stub credentials/verify (F1) that returns valid=true for any input. The SDK VCVerifier does not verify (F8). The webhook SSRF filter is incomplete (F13). The relayer is a single-key signer (SPOF). No security audit has been conducted.

Reliability: WEAK. No tests for backend or frontend. No CI. No monitoring. Indexer has transactional gaps (F12). waitForTransactionReceipt has no timeout. No graceful shutdown. No reorg handling. No retry on indexer backfill failure.

Scalability: FAIR. Postgres read model is good. RPC fallback is O(N). Contract arrays grow unbounded. Indexer is sequential per-event. Relayer is single-key. Planner is single-process. None of these are blockers for a pilot, but all would need work for production at scale.

Test maturity: GOOD for contracts (127 tests, adversarial), WEAK for off-chain. Backend has zero tests. Frontend has zero tests. SDK has light unit tests + E2E. Inspector has 5 tests (1 skip). Planner has only a benchmark. No integration tests between backend and frontend.

Maintainability: FAIR. Code is readable, but 3 GNN variants, 62KB generated abis.ts, dead code (fanOut, BullMQ, model_v2), stub endpoints, and inconsistencies (two scan_host functions, hardcoded chainId, QTRUST_REGISTRY_ADDRESS typo) raise the cost of future changes. The two prior 5,400-line meta-assessments suggest the team is spending more time on docs than on fixes.

Production readiness: NOT READY (dev-ready only). See § 18 for the full evidence list. The system can be run locally against anvil, but no aspect is ready for staging or production.

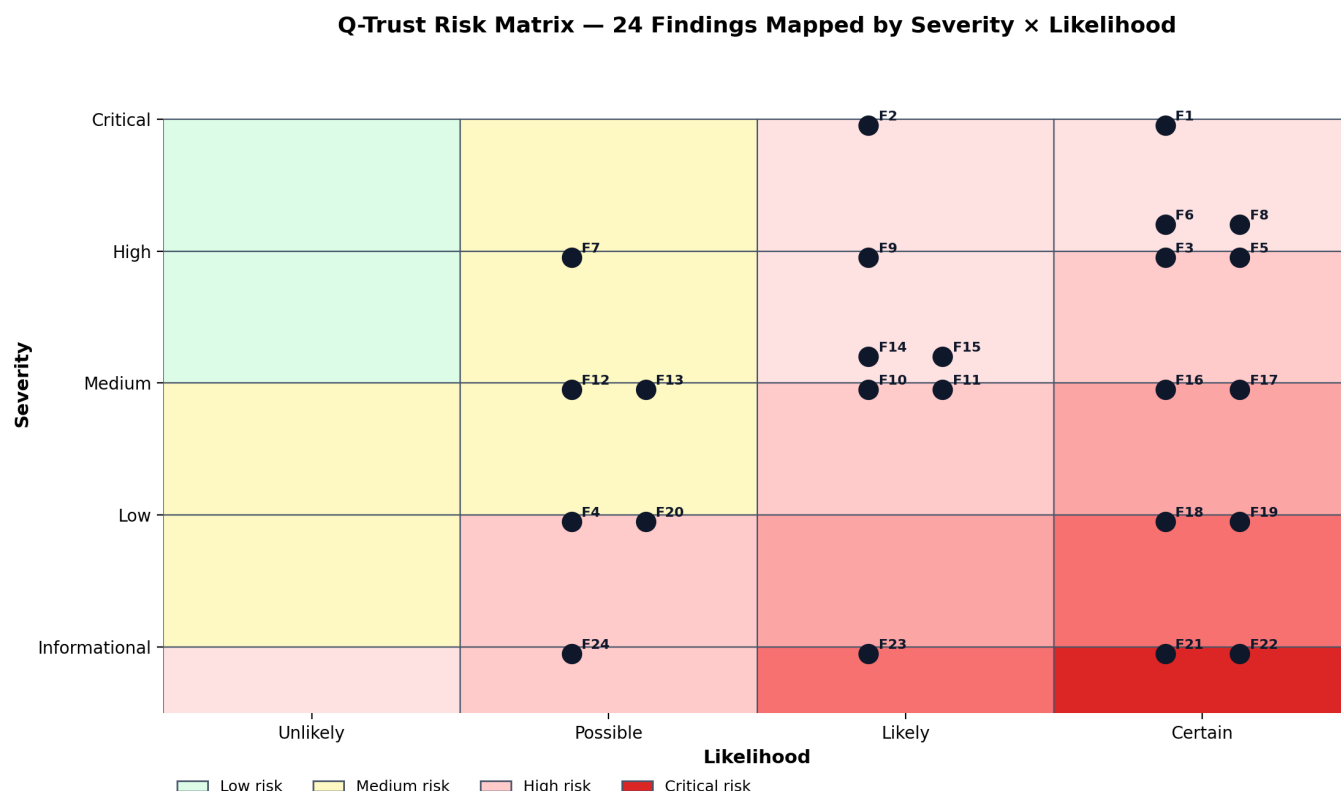


Figure 2. Risk matrix — 24 findings mapped by Severity (Y) × Likelihood (X). Critical-severity findings in the top-right quadrant (F1, F2) require immediate remediation.

Top 7 actions for the maintainers

If the maintainers can only do seven things in the next two weeks, they should be:

- **1. Close the API key dev hole** (F2, server.ts:99-102). 15 min. This is the single highest-leverage fix — it converts a critical security hole into a fail-closed gate.
- **2. Fix or remove /v1/credentials/verify** (F1, server.ts:503-521). 4h. A stub that returns valid=true for any input is actively dangerous. Either call the SDK VCVerifier (after fixing F8) or remove the endpoint.
- **3. Fix VCVerifier** (F8 + F9, sdk/qtrust/vc.py:319-379). 4h. Initialize revoked=False at the top, and actually call _ed25519_verify after extracting the public key. Without this, every VC is "valid" by default.
- **4. Wire webhook fanOut or remove the webhook system** (F3, webhook.ts:88). 4h. The current state — subscribe/unsubscribe maintain Redis sets nobody reads — is the worst of both worlds. Either wire it into the indexer onLogs callback, or remove the Redis-backed subscribe and document the env-var delivery path.
- **5. Add CI** (F17). 4h. A single .github/workflows/ci.yml that runs scripts/verify_all.sh on every push would catch F15, F9, and similar bugs before they merge. This is the highest-leverage process change.
- **6. Make EIP-712 chainId dynamic** (F10 + F11, attestation.ts:127, 233, 334, 36). 45 min. Without this, any mainnet deployment would silently break the gasless flow.
- **7. Add backend tests** (F19). 2 days. Vitest unit tests for requireApiKey, parseAssetId, the EIP-712 nonce check, and each route's input validation. Would have caught F2, F9, and the F15 cbom-before-assignment.

Audit scope statement: This audit inspected every tracked file in the repository at commit ec67f64. The contracts/lib/ submodules (forge-std, openzeppelin-contracts) were not line-by-line inspected (they are upstream OpenZeppelin/Foundry code). The planner/model.pt binary was not inspected. The notebooks were not executed. The

frontend pages beyond the entry points were not line-by-line inspected. All other source files were read in full. Findings are tagged with exact file:line citations. Where inferences were made, they are labeled as such. No finding is claimed without code evidence.